Eur. Phys. J. Plus (2022) 137:493
https://doi.org/10.1140/epjp/s13360-022-02679-7

THE EUROPEAN PHYSICAL JOURNAL PLUS

Regular Article

A gigabit TRNG with novel lightweight post-processing method for cryptographic applications

Ali Murat Garipcan[a], **Ebubekir Erdem**[b]

Department of Computer Engineering, Faculty of Engineering, Fırat University, 23119 Elazığ, Turkey

Received: 7 October 2021 / Accepted: 2 April 2022
© The Author(s), under exclusive licence to Società Italiana di Fisica and Springer-Verlag GmbH Germany, part of Springer Nature 2022

Abstract In cryptography, there is no simple methodology to obtain a True Random Number Generator (TRNG) that meets statistical requirements such as unpredictability, uniform distribution (bias) and independence (correlation) for random numbers. The statistical weakness occurring especially as a result of physical randomness and tried to be overcome by post-processing techniques is an important deficiency of TRNGs. In this study, the use of a novel lightweight post-processing technique based on chaotic substitution box (s-box) is proposed that can successfully solve this problem of TRNGs. The proposed post-processing is applied in real time to the ring oscillator-based TRNG in two different scenarios on the Altera Cyclone IV GX Field-Programmable Gate Array chip. Bias, correlation, entropy, Chi-square, Berlekamp–Massey and National Institute of Standards and Technology Special Publication 800–22 test techniques are used for statistical verification of real-time results. In addition, for the proposed method, the area–energy consumption parameters of TRNG are examined and a detailed literature (performance) comparison has been made. The acquired results indicate that the proposed method can overcome the statistical weakness problem of TRNGs by providing better trade-off than the methods in the literature. In addition, it has been observed that the performance of TRNG is high due to the advantages of the method such as simplicity, low area–energy requirement and high output bit rate. Performance and statistical analysis results confirm the cryptographic suitability of the proposed method and that TRNG can be used successfully in lightweight architecture and applications.

1 Introduction

Modern cryptographic systems are key-based algorithms. The keys for these public algorithms are random bit sequences produced from customized designs. In addition, the basic confidentiality principle of these systems is based on the assumption that for acceptable range value, these numbers are independent of their predecessors and formed by a uniform statistical distribution. In addition to these statistical properties, it is essential that the random numbers used for perfect secrecy are unpredictable and irreproducible. This cryptographic perspective also defines the characteristic behavior of the method or architectures by which random numbers are obtained [1, 2].

Random numbers to be used for cryptographic purposes are derived from two separate design classes: deterministic (pseudo) random number generators (DRNG) and true (physical) random number generators (TRNG). Any DRNG is a specific algorithm that can generate random looking numbers with good statistical properties by using randomly selected seed values. Any state value in these generators is a mathematical function of previous states. Therefore, based upon any internal state value, all future states of the generator can be easily predicted. This situation leaving cryptographic systems vulnerable to possible attacks by removing the confidentiality and complexity in the system is the most important limitation of DRNGs [3, 4].

On the other hand, TRNGs are a mechanism that often needs hardware-based solutions. Contrary to DRNGs, TRNGs that need physical noise (randomness) sources such as radioactive decay [5], thermal noise [6], jitter [7, 8], discrete [9] and continuous [10–12] time chaos and metastability [13, 14] are based on a different concept. In particular, noise sources, which are the heart of the system, are non-deterministic by nature. Unlike DRNGs whose entropy is fixed with seed value, the uncertainty in the system, i.e., entropy, increases for each random number produced in TRNGs. This providing distinctive capabilities such as unpredictability and non-reproducibility for generated random numbers makes TRNGs a powerful solution tool for sensitive cryptographic applications.

The stochastic behavior of the noise sources makes the random numbers obtained from TRNGs irreproducible. However, although the characteristic behavior of a cryptographic TRNG is well defined and understood, in practice it is not easy to obtain an ideal architecture that can generate high-quality random numbers; especially, almost all of the TRNGs in the literature suffer from the statistical weakness problem that is tried to be overcome by post-processing techniques because the entropy of noise sources from

[a] e-mail: agaripcan6223@gmail.com
[b] e-mail: aberdem@firat.edu.tr (corresponding author)

Published online: 21 April 2022

which physical trueness is tried to be obtained from very small variations is insufficient to meet cryptographic competencies. Therefore, the unpredictability (uncertainty) criterion of random numbers formed with statistically erroneous distributions is weak [2, 15].

The fact that attackers can make useful inferences by taking advantage of the weak statistical properties of keys is significant security vulnerability [16]. However, perfectly good statistical properties are needed to computationally guarantee the unpredictability of random numbers generated by TRNG for attackers. Otherwise, they cannot protect the outputs of the cryptographic applications they are used against possible attacks, because the degree of difficulty and impact of possible attacks that will be made by the attackers in order to collapse these systems directly depend on the statistical properties of the keys. Therefore, the power of the key is also decisive for the reliability of the system [4, 17].

The main purpose of a post-processing technique for TRNG is to minimize the difference between ideal and practical reality and to bring TRNG's behavior as close to the ideal as possible. In other words, it should be able to offer such good statistical competencies that would not allow attackers to do more than a blind guess on random numbers produced. Such that the random numbers whose potential weaknesses are removed by applying the post-processing should be statistically indistinguishable from an ideal TRNG output.

In today's resource-restricted modern architecture and applications in which energy consumption is major problem, there is a need for lightweight cryptographic solutions with low area energy consumption, because these architectures with limited computing and energy resources are often inadequate for cryptographic applications with high memory and time complexity. Especially with the concept of the Internet of Things (IoT), the number of users providing access to common networks and the amount of shared data are increasing exponentially. This increase also creating the need for absolute protection of information brings along the need for new security-priority solutions. In this scope, for the last ten years, it has been trying to be developed lightweight cryptographic solutions focusing on less area and energy consumption, especially of the well-known cryptographic standards in hardware. With these solutions with reduced memory and time complexity, effective results can be obtained in terms of output bit rate as well as hardware cost, because the low area/resource demand of these methods also reduces the critical path length, latency, clock cycles delay times between circuit elements [18, 19]. This case shows that lightweight solutions are also needed for physical TRNGs, because the cost problem of TRNG will directly affect the efficiency of the cryptographic applications they are used in.

Although they have impressive results, post-processing techniques are important design components causing inefficiency and cost problems for TRNGs, because complex and costly solutions offering low output bit rate to provide statistical requirements often make TRNG inefficient. Considering the high area–energy requirements and output bit rate limitations of the protocols they are used in, this indicates the need for lightweight post-processing solutions with high output bit rate for TRNG. [3, 20]. Therefore, any post-processing method is asked to produce a solution based on minimum cost as well as strong to TRNG's s statistical weakness problem.

1.1 Literature review and limitations of the available post-processing methods

For a physical TRNG, randomness refers to the outcome of a probabilistic process that produces independent, uniformly distributed and unpredictable values that cannot be reliably reproduced [21], because the basic security assumption for a TRNG is strongly related to the quality of statistical randomness as well as the fact that the generated numbers depend on a physical trueness. Otherwise, any predictability on random numbers due to poor statistical characteristic leads to a weakness in the whole system. However, TRNGs suffer from the statistical weakness problem arising as a result of physical trueness. At this point, the main purpose of the post-processing is to improve the weak statistical properties of TRNG and to prevent attackers from making strong probabilistic inferences for the generated random numbers. There are different post-processing methods in the literature recommended for this purpose. A general classification of these methods, together with the pros and cons, is shown in Table 1.

1.2 Our contribution

In this study, the hardware implementation and performance analysis of a novel chaotic s-box-based lightweight post-processing method that successfully eliminates the statistical weakness problem of TRNGs are presented. The main points contributed to the literature for the novel method are as follows:

(a) The proposed method successfully satisfies the statistical requirements needed for TRNG without sacrificing security.
(b) The chaotic s-box-based post-processing is a low-cost and highly reliable lightweight model taking up less chip area and consumes less energy.
(c) The fact that the proposed method includes s-boxes with the highest degree of non-linearity among other chaos-based architectures in the literature makes TRNG strong against advanced cryptanalysis scenarios.
(d) The performance metrics of chaotic s-boxes such as strict avalanche criterion (SAC) and bit independence criterion (BIC), which can remove bias and correlation on raw random numbers, are better than previously proposed chaos-based constructs in the literature. These mathematically guaranteed superiorities of s-boxes can provide strong statistical competencies for raw random numbers at low cost.

Table 1 A summary of post-processing methods in the literature used for cryptographic purposes

Study	Method	Methodology	Compression	Complexity	Area–energy	Advantage	Disadvantage
Ref. [22, 23]	XOR	n-bit classical XOR function	n/2 (high) ($n \geq 2$)	Low	Low	Simplicity fictional structure end easy applicability	High compression amount causing performance loss in terms of output bitrate Another important shortcoming of the method is the independence assumption, which is unlikely in practice for random numbers in physical TRNGs. Therefore, in cases where random numbers occur with strong statistical dependencies (correlation), the XOR method alone is insufficient and insecure
Ref. [24, 25]	H-function	XOR + Error correction code (ECC) based linear compression function	n/2 (high) ($n = 16$)	Low	Low	Easy applicability Entropy performance is higher than the classical XOR method	High compression amount causing performance loss Slower than XOR method
Ref. [26]	Mixing Bits in Steps and XORing Adjacent Bits (MiBiS&XOR)	XOR + substitution based confusion function	n/2 (high) ($n = 2, 3 \ldots \infty$)	Low	Low	Easy applicability More successful than XOR method in terms of entropy and autocorrelation	High compression amount causing performance loss In the proposed method, the complexity of the method, memory, and area–energy requirements increase exponentially according to the number of states (steps) of the confusion function
Ref. [27]	von Neumann (VN)	In this method, non-overlapping pairs of successive random bits are taken into account	3/4 (in the best condition)	Low	Low	Simplicity and easy applicability Ability to completely eliminate bias	The waiting time for non-overlapping consecutive bits in random number sequence may be longer than desired, and therefore, a stable output bitrate cannot be achieved for TRNGs. This further increases the compression amount of the post-processing High compression amount resulting in loss of output bitrate and entropy
Ref. [28]	N bits von Neumann (VN_N)	Von Neumann + matching table	1/2 (in the best condition)	High	High	Ability to completely eliminate bias Output bitrate increase and entropy gain due to lower compression amount than classical VN [27] method	The compression rate of method is still high For large N values, the entropy and output bitrate performance of the method can be high. But the proposed method needs more chip area for large N values and beyond $N = 17$ the efficiency of the method becomes saturated. The performance of the post-processing may vary depending on the amount of bias in the random numbers to be post-processed and whether the independence and uniform distribution assumptions are fulfilled for these numbers

Table 1 continued

Study	Method	Methodology	Compression	Complexity	Area–energy	Advantage	Disadvantage
Ref. [29, 30]	Iterated von Neumann (IVN)	Recycling discarded bits in classic von Neumann method	2/5 (average)	Low	Low	Ability to eliminate bias substantially Output bitrate increase and entropy gain property due to lower compression ratio compared to [27] and [28] Optimizability	Depending on the size of the bias, the compression amount of the proposed method, in other words the output bitrate of TRNG, is variable IVN method has the structure of a complete binary tree of n levels where n is the number of iterations. The number of levels can be increased for higher output bitrate and entropy gain. However, this situation providing a partial improvement in the output bit rate, also increases the bias and decreases the efficiency of the proposed method. Besides, the number of iterations increases the space–energy and memory requirements of the method
Ref. [7, 31]	Resilient function	[256, 16,113] Bose – Chaudhuri – Hocquenghem (BCH) based linear compression function	1/16	High	High	Strong algebraic structure Zero bias	Higher compression amount resulting in inefficiency compared to XOR and von Neumann based post-processing methods Hardware implementation requires a considerable amount of chip area
Ref. [32, 33]	Linear-feedback shift register (LFSR)	Primitive polynomial based DRNG	–	Low	Low	Low cost and Easy to implement in hardware Suitable for hardware implementation where the chip area is limited Fast and required minimal computation feature	Possibility of LFSRs being vulnerable to theoretical attacks that could pose serious security risks, as in [34] Any internal state value of the LFSR is a mathematical function of the previous states. Therefore, it is possible that extra dependencies may occur in this method, which may indicate a correlation on the output bits. The level of correlation makes it possible to model the internal state values mathematically, in other words to predict the outputs
Ref. [35]	Strong blenders	$l \times l$ right-shift superdiagonal matrices ($l = 1, 2, \ldots 256$)	–	Low	Low	Lightweight hardware architecture Compatible with all physical sources of randomness that produce more than 0.5 bits of min-entropy	In the proposed method, the input size based on the minimum entropy assumption is required to be large enough. This increases the complexity of the proposed method, that is, the hardware space–energy requirement

Table 1 continued

Study	Method	Methodology	Compression	Complexity	Area–energy	Advantage	Disadvantage
Ref. [36–38]	Hash functions	Secure Hash Algorithms (SHA)	1/16	High	High	High non-linearity degree Powerful statistical proficiency High security contribution for sensitive cryptographic applications	The number of iterative steps of cryptographic algorithms and the number of defined algebraic operations providing confusion and diffusion properties within these steps can be quite high. Therefore, the computational complexity of these post-processing methods is higher than the conventional methods in the literature. The high computational complexity increases the space–energy and memory requirements needed for the hardware-level implementation of these methods. Besides increasing the cost of production per bit of random numbers, this also makes difficult for the generator to implement on resource-constrained devices and architectures In addition to the hardware cost, the hash-based methods in [36–38] have high compression ratios
Ref. [39]	Block cipher	PRESENT cipher algorithm	–	High	High		
Ref. [40]	Block cipher	(Advanced Encryption Standard) AES cipher algorithm	–	High	High		
Ref. [3]	S-box	AES s-box matrix/table	–	Low	Low	Powerful algebraic structure Good statistical property Simple fictional structure and easy applicability Low cost	Although the proposed method includes a strong atomic basis such as AES s-box, the need for an additional input to ensure non-linearity partially weakens the performance of this method
Ref. [16]	Chaos [A]	Logistic map	f/k	High	High	High non-linearity degree Resistance to environmental fluctuations and aggressive tampering High security contribution for sensitive cryptographic applications	Although their mathematical modes are simple, the complexity of chaos-based methods in hardware realizations may be higher than cryptographic methods due to floating-point arithmetic. Depending on the type and number of circuit elements used, the chip area and energy consumption of modeled chaotic circuits are high. The cost increase caused by this is an important problem in regards to the practical usefulness of chaos-based methods In addition, due to synchronization in the methods proposed in [16] and [41], the production (sampling) rate (f) of raw random numbers in the system is reduced to the computation time (k) of the modeled chaotic systems. This situation, performing a kind of compression function for TRNG, leads to serious reductions in the output bit rate The cost of generating safe random numbers with good statistical capabilities is high in the proposed methods
Ref. [41]	Chaos based entropy pool [A]	Tent, Bernoulli, cubic and quadratic map	f/k	High	High		
Ref. [42]	Hyper chaos	Tent map based chaotic coupled map lattice (CCML)	Not reported	High	High		
Ref. [43]	Genetic algorithm	Heuristic evaluation algorithm	Not reported	High	High	Powerful confusion algorithm High autocorrelation and entropy performance	The proposed method has high complexity and is not prone to hardware implementation
Ref. [44]	Cellular Automata (CA)	3-bit one-dimensional programmable CA architecture	Not reported	Low	Low	Flexibility to be programmed with different CA architectures	The rules chosen in the proposed method are decisive for the statistical performance of the method. Therefore, choosing the ideal rule for the programmable CA architecture may require testing all the rules one by one

A: f/k is the final output bitrate of the post-processed TRNG

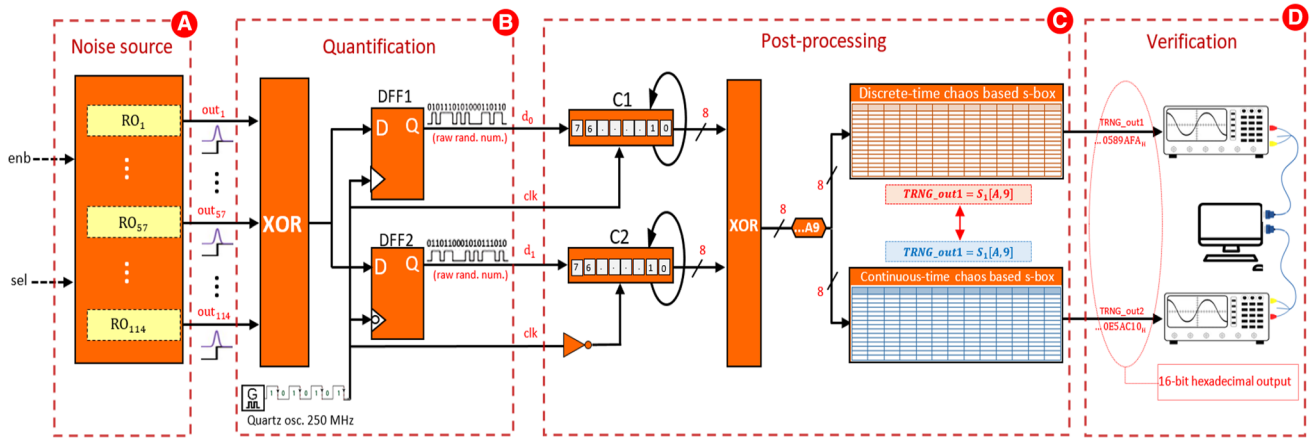


Fig. 1 General design hierarchy of TRNG for the proposed post-processing

(e) Unlike many methods in the literature, the proposed post-processing technique is compressionless and provides an output bit rate 2–4 times the sampling rate for a physical TRNG.
(f) The proposed method can provide a better trade-off between efficiency (security, output bit rate, area–energy requirement) and randomness criteria of TRNG in terms of cost and applicability compared to the methods in the literature.

1.3 Outline

The rest of the paper is organized as follows: Sect. 2 describes the theoretical details of TRNG architecture based on the proposed method. Details of hardware modeling and FPGA implementation are given in Sect. 3. Statistical analysis and hardware performance and literature comparison are given in Sects 4 and 5, respectively. The study is concluded with Sect. 6, in which the findings and future studies are briefly interpreted.

2 Theoretical and practical background on proposed method

The general structure of TRNG for the proposed post-processing method is shown in Fig. 1. This structure consists of four hierarchical components and stages: noise source, quantification unit, and the proposed post-processing technique and experimental verification. Figure 1(A) is the RO-based noise source of TRNG. This architecture consists of 114 parallel connected RO clusters, each of which consists of 3 inverters. The use of ROs as a source of randomness is much easier than complex and hardware-cost solutions using physical and analog processes. In particular, they can be easily applied on programmable logic devices such as FPGA and Application Specific Integrated Circuit (ASIC). In addition, since they do not need analog–digital conversions, their area–energy requirements and response times are low. Therefore, obtaining high oscillating clock signals is an important issue for TRNG architectures with high output bit rate [45]. This is an important factor in the choice of noise source architecture for the study.

High oscillating-independent outputs of the RO's in the system are applied as input to the quantification unit in Fig. 1B. Then, these combined outputs with XOR gate are sampled with the help of two different D-type flip-flops to obtain d_0 and d_1 outputs (oscillations) representing raw random numbers. Two rising and falling edge-triggered sampling units (D-type flip flops) are used to sample the combined oscillator outputs in the system. The sampling input and clock signal (clk) of these units are common. Although the inputs are common, the raw true random numbers generated in each cycle are characteristically different. The true random oscillation of the sampling input and the different sampling time are the main reason of this characteristic difference.

In the last step, raw random numbers are applied to the input of the chaotic s-box-based post-processing method in Fig. 1(C).The post-processing technique consists of two combiners ($C1$ & $C2$), one XOR and two dataflow designed chaotic s-box circuit elements. In the post-processing stage of TRNG, combining, XOR and substitution on s-box matrices operations are performed on raw random numbers, respectively. Each of the chaotic s-boxes in the system has 8×8 (8-bit input & 8-bit output). In other words, 8-bit (vectorial) inputs are needed to apply s-box-based substitutions on raw random numbers. $C1$ and $C2$ combining circuits are used to obtain these vectorial inputs for post-processing. The outputs of the sampling units are connected to the inputs of these circuits, and the raw random numbers at bit level generated in each cycle are combined during the 8 clock pulses. Then, these numbers reaching sufficient length are XORed before the substitution operations.

In the literature, although s-boxes are designed with nonlinear methods such as pseudo-random, heuristic and algebraic techniques, in practice they perform linear conversions between vectorial input and output bits. This behavior of s-boxes, which is similar to a reversible function, makes it possible to obtain raw random numbers containing the actual entropy in the system. This causes TRNG

Table 2 Logistic map-based s-box matrix/table

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	53	E8	49	B1	67	06	D1	02	CF	A1	37	45	07	B3	0D	F1
1	2B	23	43	E0	31	7F	8E	5B	81	1F	2E	B0	E9	B4	32	87
2	4C	42	DD	5F	AD	90	6C	DE	60	75	F7	A3	19	DC	76	C8
3	C7	74	3A	8C	10	D7	0C	24	6D	BD	B9	0E	2F	3F	AF	69
4	7B	FB	C2	9B	F6	9C	E1	9E	71	55	D9	22	D6	BA	CA	26
5	F9	2C	11	D3	63	83	17	7D	40	A0	30	1A	AA	78	8D	5A
6	FA	3E	08	34	7C	DA	CC	70	A6	FF	D2	09	B8	F3	FE	46
7	61	85	14	2A	94	04	25	29	C3	6A	E2	68	98	72	6F	39
8	3B	EB	ED	8B	C9	E3	D5	F0	1B	1D	4B	50	89	3D	21	96
9	4E	5D	AE	3C	88	EC	B7	47	EE	97	66	54	F8	EA	20	35
10	65	4A	6B	80	D4	F5	82	DF	28	92	4F	C4	03	01	48	A7
11	64	5E	A4	C1	FD	79	00	A5	BF	9A	59	52	BB	99	D8	93
12	13	D0	6E	7A	62	CE	1E	E4	36	95	5C	8F	77	41	91	84
13	CD	51	15	BC	16	1C	F4	05	FC	38	27	18	EF	73	0F	8A
14	57	33	0A	F2	E7	44	56	58	C5	9D	B2	A9	B	B5	A2	12
15	DB	4D	C6	7E	E6	9F	B6	AB	C0	BE	E5	86	CB	2D	AC	A8

to be easily manipulated or its output to be predicted with high accuracy. The XOR (modulo 2) operation is one of the most familiar and useful one-way functions in cryptography, so the purpose of the XOR operation is to hide the linear relationship between output and input that causes reversibility and to eliminate access to raw random numbers in the system.

The most significant component of the proposed post-processing technique is the chaos-based s-boxes proposed in Ref. [46]. S-box architectures, in which discrete (Logistic and Henon map)- and continuous-time (Chen and Lorenz system) chaotic systems are used as the randomness source, stand out with their powerful cryptographic features. In the study, more than 20,000 random s-boxes are generated for different chaotic system classes. The s-boxes obtained from these chaotic systems are optimized according to cryptographic performance metrics such as the highest non-linearity and the smallest XOR distribution table, and the best solution is achieved. In fact, the s-boxes produced have the highest non-linearity degree among other chaos-based structures in the literature. Also, chaotic systems with the best performance in terms of randomness source are preferred and care is taken to select the most appropriate initial and control parameters for these structures. This increases the randomness in the system and makes the design methodology of s-boxes cryptologically strong. Moreover, generated s-box structures are better than previously proposed s-box structures for SAC and BIC criteria. These mathematically guaranteed superiorities of S-boxes in terms of security and randomness have been decisive in the choice of post-processing.

The s-boxes for the logistic map and Lorenz system are shown in Tables 2 and 3. These s-boxes are applied to the TRNG architecture in two different post-processing scenarios to improve the statistical randomness quality of random numbers. The s-box tables used are decisive for these scenarios and each scenario contains a discrete- and continuous-time s-box table. In the first scenario, Logistic map—Lorenz system and in the second scenario Henon map–Chen system s-box tables are applied together. Other chaotic s-boxes used for this scenario are given in Appendix A. Ref. [46] can be checked for detailed design criteria and other performance metrics of s-boxes.

3 FPGA implementation

FPGAs are popular digital logic devices that can be applied to cryptographic systems as a whole. The characteristics properties of these devices such as high output bit rate (a), parallel operation (b), re-configuration (c), platform independence (d), low cost (e), security (f), easy design cycle (g), easy integration (h) and low power consumption (i) can provide significant facilities in solving complex cryptographic problems; especially, it is an important security requirement that FPGAs can provide the opportunity to implementation on a secure computing zone for cryptographic studies, in contrast with vulnerable digital environments open to the advanced attacks. Therefore, the inability to physically obtain the device does not allow the application of effective cryptanalysis techniques with advanced side channel support such as cache memory, timing, dynamic power consumption, electromagnetic radiation measurement. This is a serious difficulty for possible attacks. Moreover, physical randomness resources can be obtained easily and quickly on these devices without the need for analog–digital conversions for TRNGs. This also makes it possible to integrate TRNG into the cryptographic systems easily [47].

Altera Cyclone IV GX FPGA chip is used as the digital implementation platform of the study. Very High-Speed Integrated Circuit Hardware Description Language (VHDL) language is used for hardware modeling of TRNG. Quartus Prime Web Edition

Table 3 Lorenz system-based s-box matrix/table

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	5C	1D	A9	D4	B3	72	98	BF	B8	E7	46	C3	65	6A	0C	A4
1	CC	22	CF	27	B2	2A	85	F1	D7	D9	C9	01	EE	9D	C2	B9
2	58	9B	6C	30	F3	38	32	6D	21	AC	E8	DE	FA	06	34	49
3	2B	A8	26	25	61	2F	3A	A1	B5	87	B1	A0	DC	7C	4B	11
4	50	4D	89	D5	CB	91	D1	20	94	CE	EC	6E	33	51	F4	74
5	B0	81	E1	A6	FE	3C	C6	76	A3	04	3D	42	13	0E	5E	A5
6	1B	18	7E	C0	92	E4	C5	0D	0A	D8	DD	EF	7B	52	E2	FF
7	23	AF	88	7A	AB	3F	CA	B4	29	39	EA	AA	E3	F0	67	A7
8	4C	00	F8	31	62	8A	17	DA	6B	8D	BA	8F	03	70	BB	15
9	E9	AE	3B	08	75	9F	ED	DF	BD	FB	1C	4A	2D	4E	0F	3E
10	73	07	54	44	E6	96	8C	F9	83	0B	CD	24	79	66	41	5B
11	9C	14	78	99	9E	09	02	2C	84	D0	77	7F	F2	93	57	28
12	D6	FD	8E	B7	47	1F	DB	E0	69	37	16	10	56	5A	53	45
13	D2	60	36	B6	6F	C4	82	80	BE	C8	55	48	1A	86	95	97
14	59	F5	05	8B	64	BC	5D	71	EB	C7	AD	19	63	90	1E	F7
15	9A	2E	E5	68	43	4F	C1	D3	40	35	FC	5F	12	7D	A2	F6

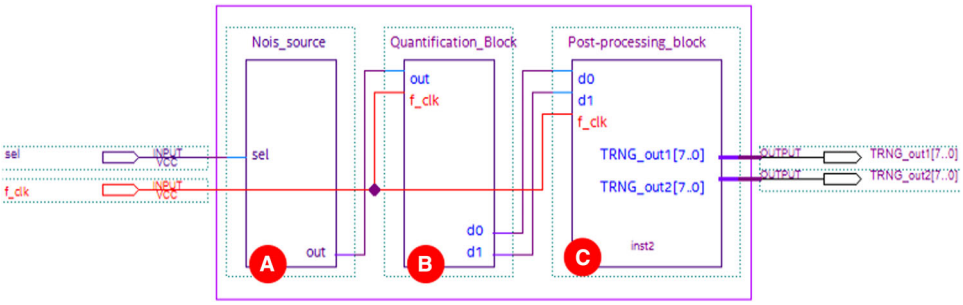


Fig. 2 Top-level representation of TRNG in Quartus

17 platform is used for behavioral and dataflow descriptions of the modeled components with real-time simulation, synthesis and place routing of the system. TRNG architecture has been tested in two different scenarios for the proposed chaotic s-box-based post-processing technique, and successful results have been obtained. The simplified top-level block representation of hierarchical components representing TRNG is shown in Fig. 2. Details of the hardware modeling of these simplified block circuit elements are given in Fig. 3.

Figure 3A is the RO-based noise source of the TRNG, which is given a block representation in Fig. 2A. The noise source consists of 114 parallel connected RO cores with common trigger pins. Figure 4 shows the combinational definition and behavioral (hardware) modeling of an RO circuit, whose block representation is given in Fig. 3. In Fig. 4b, one 2×1 multiplexer (mux) and a block circuit element formed by three sequentially connected inverters are used for modeling the core architecture. In Fig. 4b), mux is used as the control variable of the RO instead of the NAND gate. The $data0$ and $data1$ pins of this mux are the enable (enb) and feedback inputs of the RO, respectively. The selection pin (sel) of the mux is the user-controlled trigger signal of the oscillators obtained from the physical environment. This pin controlled with a slider switch on the FPGA is also the start/stop button of TRNG. Such that, for $sel = ^{'}0^{'}$, enable end of mux connected to $data0$ input is active. The $data0$ pin is connected to + VCC, and since the oscillator cycle consists of an odd number of sequential inverters, the oscillator output is fixed at the logic '0' level in this case. To obtain the clock jitter at the oscillator output, the $data1$ (feedback) input of mux must be active. Therefore, the sel pin is set to the logic '1' state and remains in this position throughout the operation of the TRNG. In Fig. 3, sel and $data0$ inputs of the muxes in parallel-connected oscillator inputs are common for synchronization.

In Fig. 3, the high oscillating outputs of the RO's connected in parallel are combined with the XOR process. These outputs, which are then reduced to a single non-periodic input (f_{np}), are sampled with the help of rising and falling edge-triggered two different D-type flip flops (DFF1 and DFF2). At this stage, the frequency of the clock signal used for sampling is 250 MHz. Bit-level raw random numbers are obtained separately for the rising and falling edge transitions of this clock signal. These outputs (d_0 and d_1) sampled in two different phases represent raw true random numbers in the system at bit level with poor statistical quality. Since two-bit raw random number is obtained in each sampling cycle in the system, the production rate of these numbers is twice the sampling rate, i.e., 500 Mbit/s. Then, these numbers are applied as an input to the proposed post-processing method in Fig. 2C.

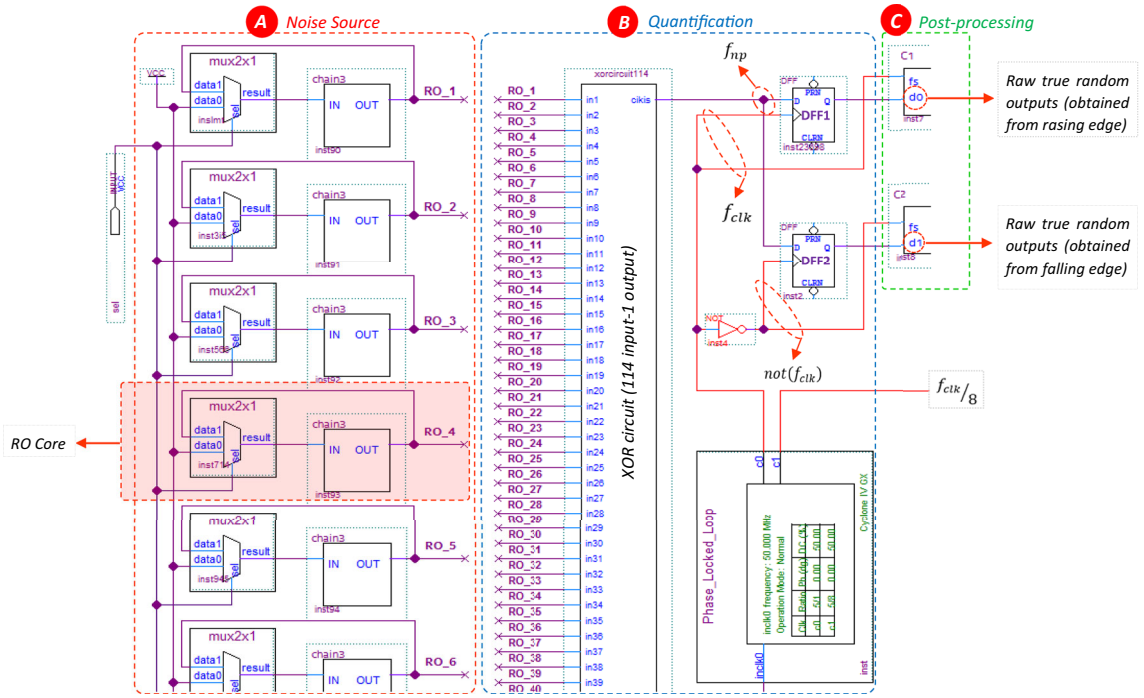


Fig. 3 RO-based noise source of TRNG

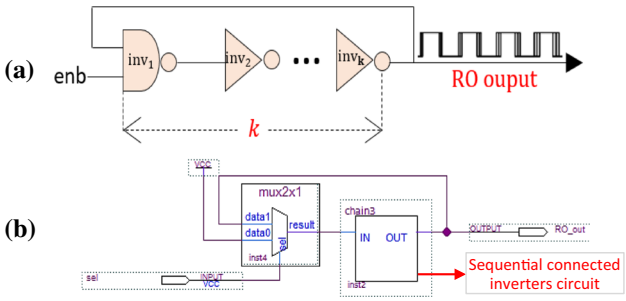


Fig. 4 **a** Combinational and **b** hardware modeling of RO core

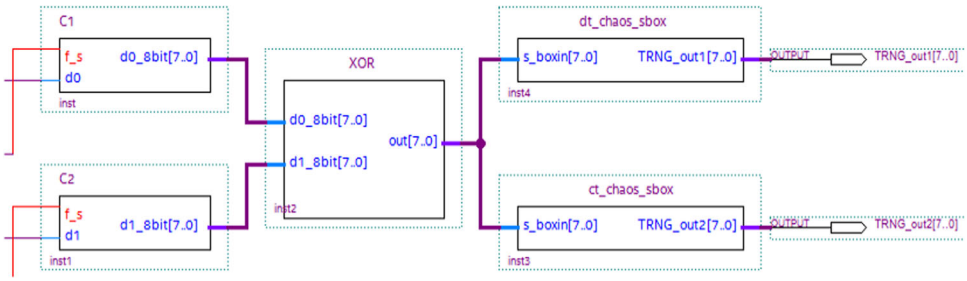


Fig. 5 Hardware modeling of the proposed post-processing method

3.1 Hardware modeling of the proposed post-processing method

Quartus modeling of the post-processing technique given in Fig. 5 consists of two combiners ($C1\&C2$), one XOR and two different chaos-based s-box circuits (dt_chaos_sbox & ct_chaos_sbox). Raw random numbers are first applied to the inputs of the combining circuits in the post-processing. These circuits can also be thought of as a 3-bit (7–0) down counter-controlled 8-bit register. The placement of one bit of raw random numbers intended to be merged into registers is form the most significant bit (msb) to the least significant bit (lsb). In other words, the first and last bits of each 8-bit sequence of raw random numbers generated are assigned to the most and least significant bit values of the register, respectively. The decreasing values of the counter with each sampled random number also address to which bit of the register this number will be assigned. With each new cycle (every 8 clock pulses), the counter is reset and the register contents are replaced by new generated random numbers. These operations occur synchronously in two different combining circuits for two different sampling times.

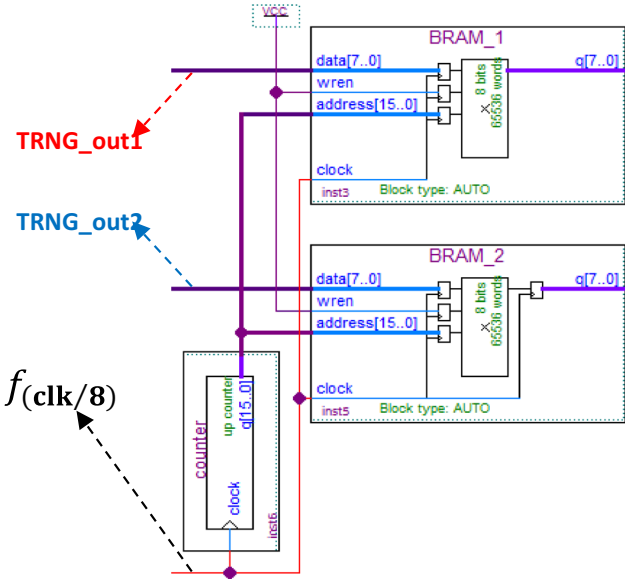


Fig. 6 BRAM architectures

The outputs of the combiner circuits in Fig. 5 are also the inputs of the 8-bit dataflow XOR circuit. Therefore, the combined vectorial inputs obtained in every 8 clock pulse are XORed synchronously with each other. The outputs of the XOR circuit are synchronously applied to the input of two different chaotic s-box tables. Although the input value is the same, the row and column values are different for the 8-bit s-box matrix values between 0 and 255. Therefore, outputs of 8 bits replaced in two different s-boxes for the same inputs are also different from the each other.

The proposed post-processing is applied on TRNG according to two different s-box scenarios. The post-processing technique in each scenario consists of two discrete and continuous chaos-based s-box matrices. Logistics map and Lorenz system are used in the first scenario, and Henon map and Chen system s-boxes are used in the second scenario. Substituted outputs for these scenarios are calculated as follows: The first four bits of the 8-bit input vector are taken as the row and the last four as the column value of the s-boxes. Then, the s-box values equaled to the intersection of the relevant row column are transferred to the output. The two separate 8-bit outputs of the post-processing ($TRNG_out1$, $TRNG_out2$) are also the 16-bit true random outputs of the TRNG.

3.2 Other implementation details

The 8-bit outputs of TRNG have been saved in two separate block memory architecture (BRAM) in Fig. 6 for statistical analysis and real-time verification. BRAM, whose column width is 8 bits, consists of 65,536 rows. A common up counter circuit of 16 bits is used for row addressing in the system. Clock inputs for counter and BRAMs are common for synchronization. The frequency of this signal is reduced to 1/8 of the sampling clock signal, that is, $250/8 = 31.25$ MHz. In other words, the address values of the counter will increase simultaneously with the acquisition time of TRNG's 8-bit true random outputs. Thus, the random numbers generated in each cycle will be stored in the correct memory locations without loss. Discrete- and continuous-time chaotic s-box outputs in the system are connected to the data inputs of the $BRAM_1$and$BRAM_2$ blocks in Fig. 6, respectively.

The FPGA chip used for implementation has three different crystal oscillators on itself that can generate 50 MHz clock signals. The device allows these reference clock signals to be obtained at higher or lower frequencies with the help of a phase-locked loop (PLL). The Altera PLL IP core circuit element supports six different clock feedback modes. Each mode allows clock multiplication and division, phase shifting, and duty-cycle programming operations on the reference (input) clock signal. Therefore, PLL is used to obtain the low and high frequency clock signals needed for sampling and storing processes in the system.

3.3 Real-time simulation results

Real-time simulation results of the Logistic map and Lorenz system-based s-box post-processing scenario of TRNG are shown in Figs. 7 and 8. Since the RO outputs are fixed at the logic '0' level for $sel =$ '0', the combined 8-bit s-box inputs in the system are fixed at the level of "00,000,000." For these fixed inputs, the outputs of TRNG remain constant at the $s-box1[0,0]_{hex} = [53]_{\text{hex}}$ and $s-box2[0,0]_{hex} = [5C]_{\text{hex}}$ level. In this state also accepted as the initial position for TRNG, the simulation results are shown in Figs. 7 and 8 (a). $sel =$ '1' state is the working position for TRNG. For this position, feedback loop of parallel connected RO's is active. In this state where physical randomness is obtained, the TRNG outputs are true random. These outputs are simulated in Figs. 7 and 8b according to the state of the chaotic s-boxes. The production rate of raw random numbers in the system is equal to

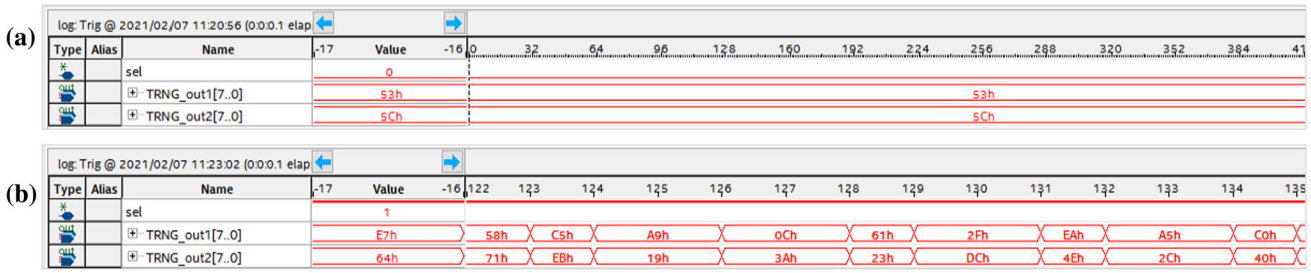


Fig. 7 Real-time simulation results of TRNG for **a** initial and **b** run-time positions in the first scenario

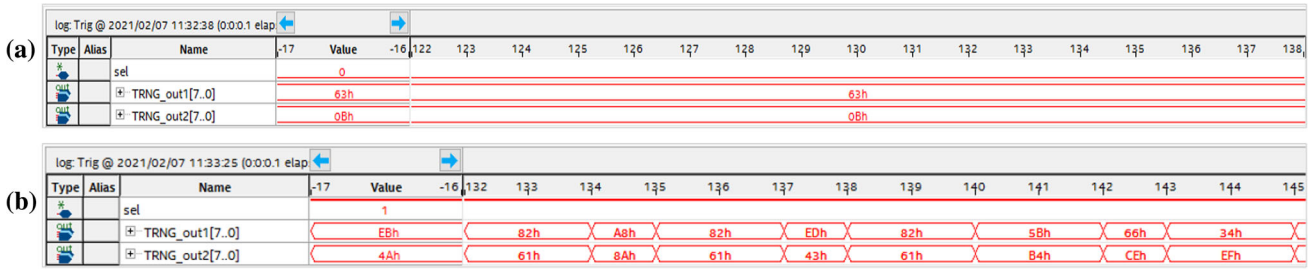


Fig. 8 Real-time simulation results of TRNG for **a** initial and **b** run-time positions in the second scenario

twice the sampling rate. The proposed post-processing technique not causes any loss by compression on these numbers. Therefore, the output bit rate of TRNG is 500 Mbit/s in two separate scenarios.

4 4. Experimental verification

In this section of the study, the main purpose is to analyze the statistical contribution of the proposed method on TRNG from a cryptographic point of view. In the analysis process, besides bias, correlation and entropy tests, Chi-square and NIST SP 800–22 statistical randomness evaluation tests are used. In addition, statistical complexity measures of random numbers have been analyzed by Berlekamp–Massey algorithm. The successful statistical results achieved in this section confirm the basic idea of the study.

In both scenarios, the 16-bit output of TRNG consists of two separate 8-bit outputs. These independently analyzed outputs are also discrete- and continuous-time chaotic s-box outputs. The main reason for analyzing these outputs independently is to correctly analyze the contribution of discrete and continuous time s-boxes on raw random numbers. This case allowing the opportunity to compare the results with each other will also provide useful information in terms of the validity of the method proposed in [46]. BRAM contents in which real-time outputs of TRNG are recorded are used for statistical analysis. These contents have been obtained and transformed through the Joint Test Action Group (JTAG) interface.

In this section, 120 different test files are used for each post-processing scenario: 40 of these test files consist of raw random numbers and 80 independently analyzed 8-bit post-processed TRNG outputs. In the confidence interval analysis of NIST SP 800–22, 1000 different test files are used for each post-processing scenario. Each analyzed test file consists of 524,288 (65,536 × 8) bits.

4.1 Bias analysis

The reliability of random numbers in cryptography is based on the assumption that the elements of these numbers occur independently of each (uncorrelated) other and with an equal occurrence probability (unbiased) of 1/2 in a defined range. The deviation amount of the occurrence probability from this ideal value can provide sufficient evidence to decide that any number sequence is not cryptographically random. Monobit test is used for bias analysis in this subsection of the study. With the Monobit test, the probabilistic distributions of raw and post-processed number sequences and their closeness to the ideal value are analyzed in terms of statistical randomness.

Monobit (frequency) test is used to examine the probabilistic distribution of bit-level random numbers. This test technique is also used in the NIST SP 800–22 test suite. The main focus of the Monobit test, in which the half-normal distribution is taken as reference, is to determine the proximity of the frequency of ones/zeros to the expected ideal value of $n/2$ in the n-bit random number sequence. [48, 49]. Mathematical definition of Monobit test is as in Eq. 1–3, where $S = x_0, x_1, x_2 \ldots x_{n-1} n$ bit random number sequence. In Eq. (1) and (2), S_{dif} and S_{obs} are the cumulative difference calculated for the random number sequence and the observed p_{value} calculated for this difference value, respectively. The real p_{value}, in other words the test statistic, is calculated as in Eq. (3), with $erfc$ complementary error function. Monobit test is hypothesis based, and the statistical significance level (a)

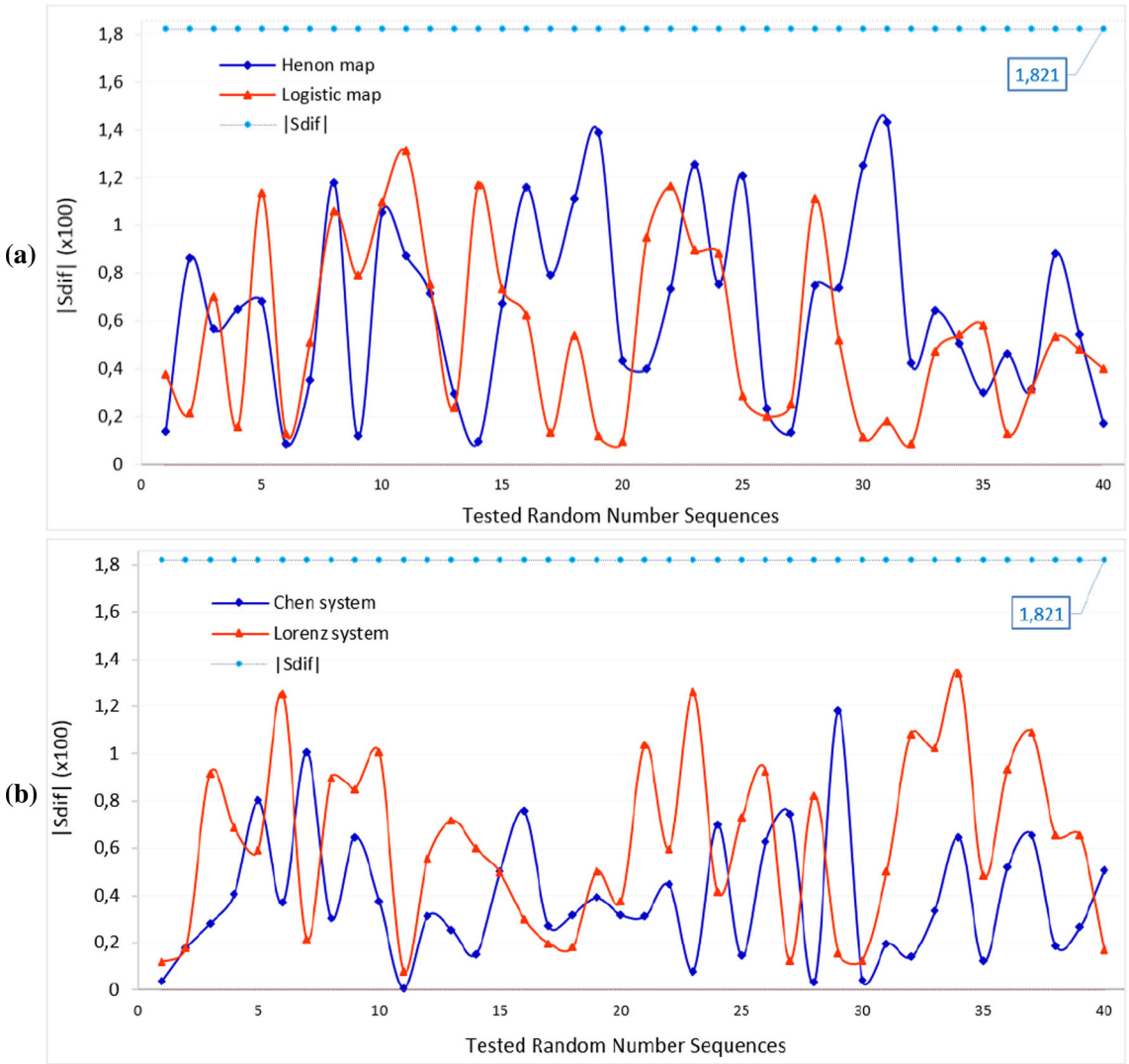


Fig. 9 Frequency test results of TRNG for the **a** first and **b** second scenarios

for this test is 0.01 as in other NIST SP 800–22 randomness tests. In order for the test to be considered successful, the $p_{value} \geq a$ criterion must be met.

In the case of $a = 0.01$, this equation shows us that the maximum difference between the total number of 1's and 0's in a random number sequence of 524,288 (n) bits can be 1821. Therefore, in order to meet the $p_{value} \geq a$ requirement, that is, to control the bias in random number sequences, the total difference between array elements (S_{dif}) must be in the range of $0 \leq |S_{dif}| \leq 1821$. In cases where the number sequences are formed with higher cumulative difference values such as $|S_{dif}| > 1821$, the Monobit test is considered unsuccessful because $p_{value} < a$. In this case, cannot be mentioned randomness for the number sequences tested. For $|S_{dif}| = 0$ where the p_{value} is equal to 1, the random numbers are assumed to occur with an ideal statistical distribution of 1/2 and are perfectly random. Monobit test results of TRNG for post-processing scenarios are shown in Fig. 9.

$$S_{\text{dif}} = y_1 + y_2 + y_3 \ldots + y_n \text{ ve } y_i = 2x_i - 1 \tag{1}$$

$$S_{\text{obs}} = \frac{|S_{\text{dif}}|}{\sqrt{n}} \tag{2}$$

$$p_{\text{value}} = erfc(z) = erfc\left(\frac{S_{obs}}{\sqrt{2}}\right) = \frac{2}{\pi}\int_{z}^{\infty} e^{-u^2} du \tag{3}$$

In both scenarios, all of the cumulative difference values calculated for raw random numbers before the proposed post-processing are far above the acceptable limit value. The sampling rate of TRNG for the suggested post-processing is 250 MHz. The cumulative difference values calculated for raw random numbers for the same sampling rate are in the range of 45–48 thousand. These cumulative

values increasing depending on the sampling rate also cause the trade-off problem between the output bitrate of TRNG and the randomness quality, whereas these difference values are at the level of 22–25 thousand for 50 MHz sampling rate.

After the post-processing is applied, the cumulative difference values calculated for substituted outputs in both scenarios are normalized (reduced) below the limit value as shown in Fig. 9. Therefore, the test results are successful as the condition for $p_{\text{value}} \geq a$ will be met for these difference values. Despite the high sampling frequency, the ability of the proposed post-processing to successfully remove bias is an important proof that the proposed post-processing is cryptographically strong. It can be said that design criteria such as SAC and BIC of s-boxes, which provide a balanced distribution especially for substituted raw random outputs, are important factors in the success of the Monobit test.

4.2 Autocorrelation analysis

The autocorrelation test technique proposed in [49] has been used to show that random number sequences do not have any internal statistical dependence on themselves. The mathematical definition of this test technique is as in Eq. 4–5. The test calculates the autocorrelation coefficient (X_5) between any random number sequence $[x_0, x_1, x_2 \ldots x_{n-1}]$ and the sub-sequences obtained by shifting this sequence for different integers d in the range $[1, (n/2)]$. In Eq. (4), $\oplus$ denotes XOR operation.

$$A(d) = \sum_{i=0}^{n-d-1} b_i \cdot b_{i+d} \tag{4}$$

$$X_5 = \frac{2[A(d) - (n-d)/2]}{\sqrt{(n-d)}} \tag{5}$$

In test technique, the $-Z_{a/2} \leq X_5 \leq Z_{a/2}$ condition must be fulfilled in order to make the independence assumption for random numbers. In the equality, Z_0 is the probability density function of a normally distributed random variable with mean (μ) 0 and variance (σ^2) 1. $Z_{a/2}$ values are calculated on the standard distribution table, where a is the significance level. Two different significance levels are selected as $a = 0.1$ and $a = 0.05$ for the independence assumption in the test criteria. The max $Z_{a/2}$ values defined on the standard distribution table for these values are 1.649 and 1.96, respectively. Autocorrelation test results of TRNG calculated for $d = 8$ value are shown in Fig. 10.

In both scenarios, X_5 values calculated for raw random numbers do not meet the conditions of $-1.645 \leq X_5 \leq 1.645$ and $-1.96 \leq X_5 \leq 1.96$. Therefore, these results are not sufficient to make an assumption of statistical independence for raw random numbers. X_5 values below the acceptable threshold indicate the presence of strong negative correlation between raw random numbers. With the applying of the post-processing technique, X_5 values calculated for the same d values have been reduced to the acceptable range. Especially the results obtained for $a = 0.1$ and $a = 0.05$ significance level can be said that TRNG produces random numbers independently at 90% and 95% confidence interval, respectively. It is clearly seen that the proposed post-processing for the same results removes statistical dependencies on raw random numbers. For powerful s-box architectures, it is desirable that the statistical dependency between output bits is as small as possible. BIC criterion that can eliminate the correlation between input and output bits is a significant design criterion that must be fulfilled for s-boxes. Therefore, the use of s-boxes in the post-processing stage is a major factor in the performance of the test results confirming the independence assumption for tested random numbers.

4.3 Chi-square analysis

In the study, in addition to NIST SP 800–22, Chi-square test technique is applied to random numbers as another statistical randomness test technique. Mathematical definition of the Chi-square test is given in Eq. (6). In the equality, k is the class (group) number. O_i and E_i are the expected and observed frequency values for calculated the i. class, respectively. In an ideal random number sequence of size n, the expected frequency distribution of each group within this sequence should be n/k. The sum of squares of the difference between observed and expected frequency values for each class $(1, \ldots k)$ in a distribution is the Chi-square statistic ($X_0{}^2$). The distribution of the test statistic is also the Chi-square distribution characterized by $k - 1$ degrees of freedom [12, 49, 50].

$$X_0^2 = \sum_{i=1}^{k} \frac{(O_i - E_i)^2}{E_i} < X^2_{[a,k-1]} \tag{6}$$

Since the outputs of TRNG are at hexadecimal (hex) level, the k (group) number is naturally 16. Since hex numbers between 0 and 15 are encoded with 4 bits, the length of the tested sequences is 131072(524288/4). Therefore, the expected frequency values of each class for this length of number sequences will be 8192(131072/16). The calculated Chi-square test results of TRNG for both scenarios are shown in Fig. 11. The results are obtained according to two different significance (reliability) levels as $a = 0.01$ and $a = 0.005$.

The limit values of $X^2_{[a,k-1]}$ corresponding to significance levels are 30.578 and 32.801, respectively. These values are obtained from the Chi-square distribution table. Therefore, the condition $X_0{}^2 < X^2_{[a,k-1]}$ must be fulfilled in order to conclude that the tested random number sequences are uniformly distributed. In Fig. 11, it is clearly seen that this condition is not fulfilled in both scenarios for the test results of raw random numbers. Unsuccessful results show that raw random numbers do not occur with a

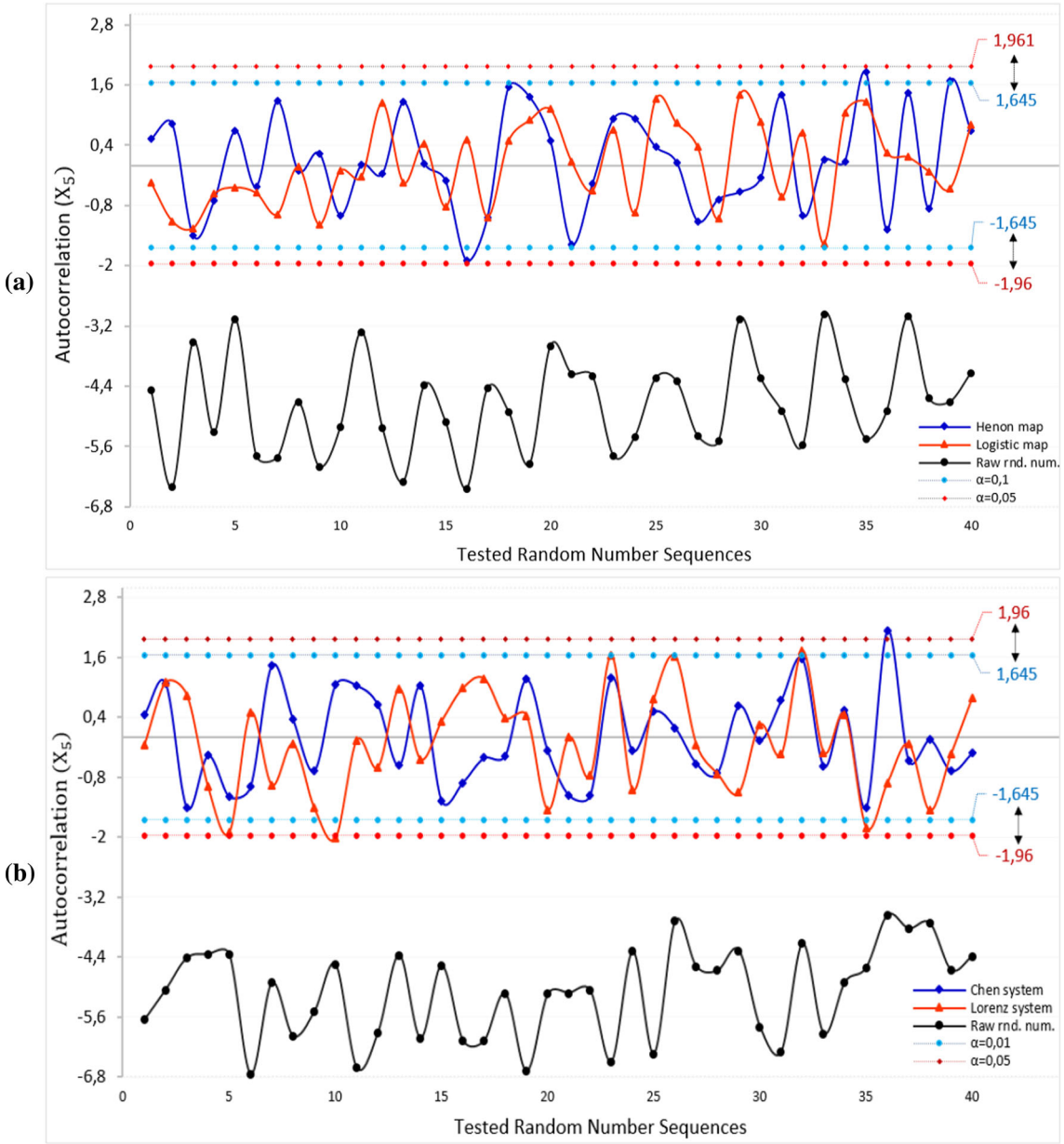


Fig. 10 Autocorrelation test results of TRNG for the **a** first and **b** second scenarios

uniform distribution within the defined range. Figure 11 shows that the high Chi-square values calculated for raw random numbers are normalized to below the limit value after the post-processing is applied. These normalized results confirm that the proposed post-processing provides a uniform distribution for tested random numbers and that these numbers are statistically random.

4.4 Linear complexity analysis

Berlekamp–Massey algorithm is used for linear complexity analysis of post-processed random numbers. The linear complexity measure of random numbers can provide valid proof of whether these numbers are produced by true random methods. Berlekamp–Massey algorithm calculates the size and feedback polynomial of the shortest length LFSR that can produce the tested number sequence. For the algorithm, the complexity of the random number sequence s^n should be $L \leq \frac{n}{2}$, where L is the linear complexity value and n is the sequence size. In other words, an LFSR of size $n/2$ is needed to generate an n bit random number sequence obtained from an ideal TRNG. Values of L close to $n/2$ indicate that random numbers have a high degree of nonlinearity (complexity) [42, 49]. This also provides strong evidence that the number sequences tested are generated randomly. Otherwise, randomness cannot be mentioned and the number sequences tested are considered to be predictable.

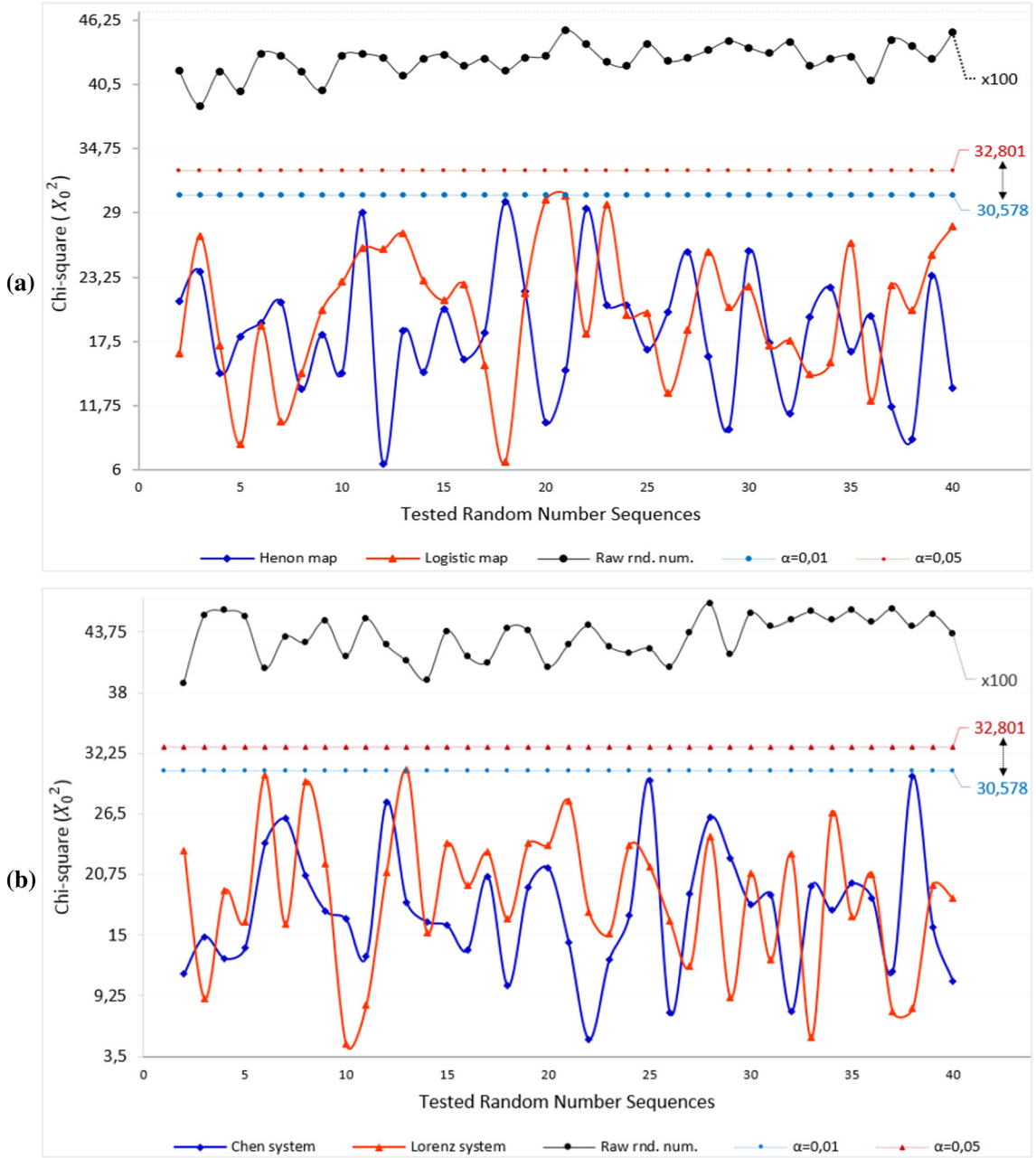


Fig. 11 Chi-square test results of TRNG for the **a** first and **b** second scenarios

The running time of the Berlekamp–Massey algorithm for determining the linear complexity of a binary sequence of bit length n is $O(n^2)$ operations [49]. For this reason, only the first 100,000 bits of random numbers is given as an input to the algorithm and the results in Fig. 12 are obtained. The results showing the high linear complexity of the tested numbers also confirm the true randomness of these numbers. The same results also show that the proposed post-processing method does not have a negative effect on the true randomness in the system.

4.5 Entropy analysis

The concept of entropy, also known as Shannon entropy in information theory, is considered a measure of the unpredictability of a random variable. Shannon defined the expected value for the possible states of the random variable with the relation in Eq. (7). In Eq. (7), $p(x_i)$ represents the occurrence probability of x_i where $x = [x_0, x_2 \cdots x_{n-1}]$. The H_x value varying in the range of $[0 - 1]$ in Eq. (7) takes the maximum value as the theoretical result of the situation when the number sequence occurs with a

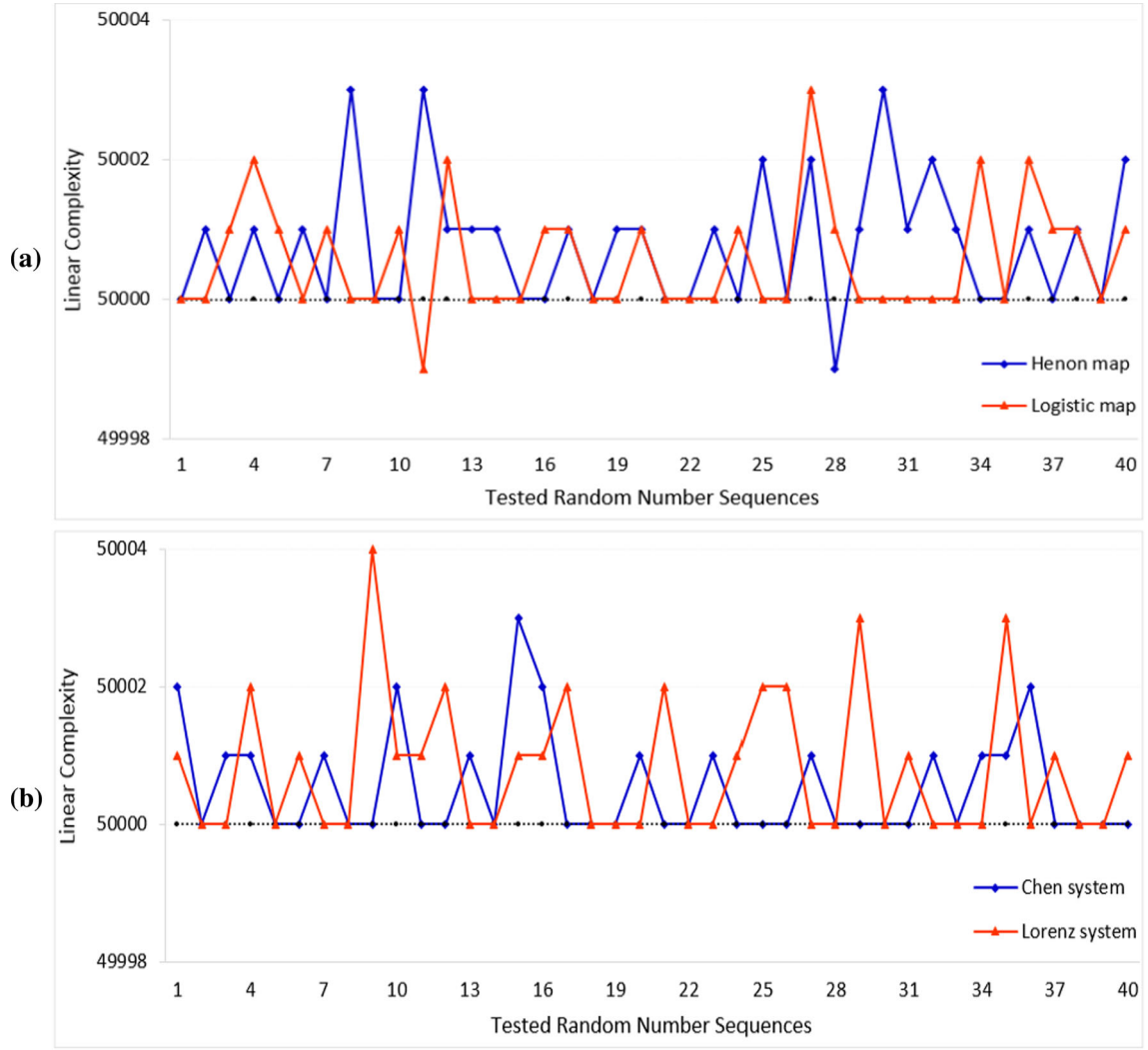


Fig. 12 Berlekamp–Massey results of TRNG for the **a** first and **b** second scenarios

uniform distribution that is equal to 1. In cases where the results are definitely known, $H(x)$ is equal to 0 and cannot be mentioned randomness or uncertainity in these cases [33, 51]. Shannon entropy test results of TRNG are shown in Fig. 13.

$$H_x = -\sum_{i=0}^{n-1} p_{x_i} . \log_2\left(p_{x_i}\right) \tag{7}$$

The high entropy for random numbers makes these numbers resistant to cryptoanalysis scenarios. This resistance also prevents attackers from making useful inferences for high accuracy estimation of these random numbers. Therefore, the entropy of random numbers is an important indicator that a TRNG is cryptographically strong. When the results in Fig. 13 are examined, it is seen that the entropy for raw random numbers cannot exceed a certain value (max. 0.956). When the same results are examined with regards to the post-processing, it is seen that entropy is very close to the ideal value. The high entropy for random numbers depends on the statistically uniform distribution of these numbers. Entropy test results are consistent with the Chi-square test results confirming that random numbers are formed with a statistically uniform distribution in Fig. 11. Entropy test results show us that the proposed post-processing increases the entropy per bit, reducing the possible impact of attackers on random numbers. The ability of the post-processing to provide uniform distribution for random numbers increases the uncertainty in the system and the unpredictability of random numbers. This is an important factor for entropy test results close to the ideal value in Fig. 13.

4.6 NIST SP 800–22 randomness test

It is imperative to guarantee that the TRNG has good statistical properties using statistical test tools. That is required for directly using the TRNG outputs in cryptography. There are different known test techniques/tools in the literature such as FIPS 140-1,

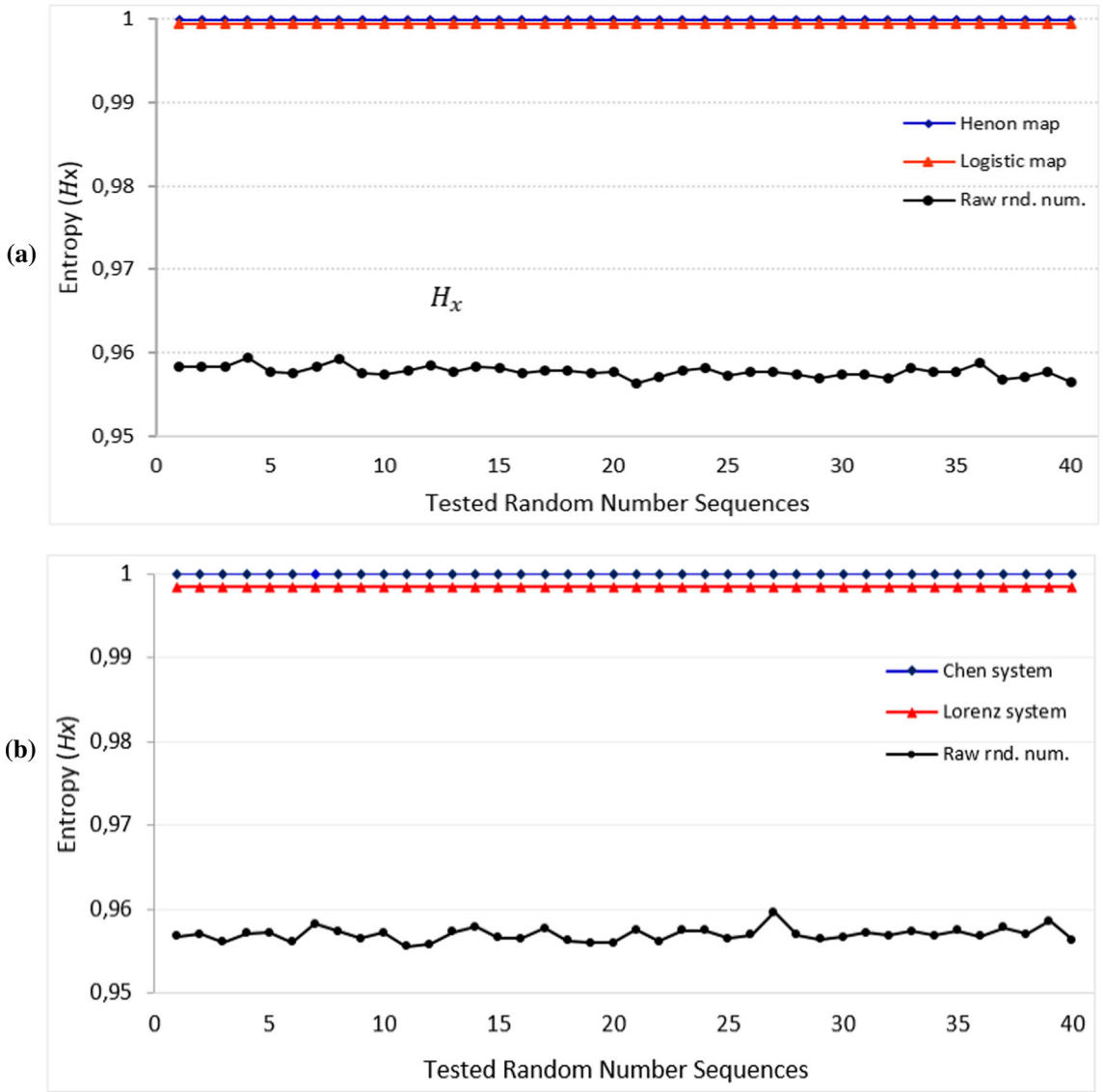


Fig. 13 Shannon entropy test results of TRNG for the **a** first and **b** second scenarios

DIEHARD, ENT, AIS31 and NIST SP 800-22 [48] developed for this purpose. These tests are usually hypothesis-based and use specific parameters to test the null hypothesis such as H_0. The acceptance or rejection of the H_0 hypothesis can provide valid evidence for the uniform, patternless and unpredictability characteristics of the tested random numbers [42, 52].

The NIST SP 800–22 statistical randomness test suite is a frequently preferred and highly valid test tool with its mathematical infrastructure, large number of tests, length of the input set and adaptability. Test suite consists of 15 different subtest criteria. The success of each test criterion depends on two parameters, p_{value} and a. The p_{value} in the range $[0-1]$ is the randomness statistic of the sequence/numbers tested. The parameter a supports the assumption that the number sequence is not random, and also corresponds to the typical significance level for the test criteria. This parameter is generally chosen as 0.01 for cryptographic purposes. In any test criterion, if $p_{value} \geq a$, the H_0 hypothesis is accepted. The ability to make the assumption of randomness for the number sequences tested depends on the fulfillment of this condition for all test criteria. Otherwise, the H_0 hypothesis is rejected and the assumption that the numbers are not random is accepted [48, 51].

NIST SP 800-22 tests are applied in two stages within the study. In the first stage, $p_{value} \geq a$ distributions of random numbers for each criterion are examined for each test criterion. In the first stage, these results confirming that the random numbers are formed with a statistically uniform distribution are shown in Table 4. The results are also consistent with the Chi-square, bias and entropy test results in terms of statistical randomness and uniform distribution.

In the second stage, 1.000 different random number sequences are used and the confidence interval of each test criterion is determined. The confidence interval of any test criteria in NIST SP 800–22 is determined as in Eq. (8). In the equality, $\widehat{p} = (1 - a)$

Table 4 NIST SP 800–22 statistical randomness evaluation test results of TRNG

Test Name	Setting Parameters	TRNG, without post-processing		TRNG, with proposed post-processing				
		p value	Result	Scenario 1		Scenario 2		Result
				Logistic map	Lorenz system	Henon map	Chen system	
Frequency test		–	Fail	0.901	0.807	0.948	0.960	Success
Frequency test within a block	M = 128	–	Fail	0.916	0.730	0.423	0.939	Success
Run test		–	Fail	0.566	0.178	0.486	0.742	Success
Test for the longest run of ones in a block		–	Fail	0.875	0.683	0.145	0.939	Success
Binary matrix rank		0.840	Success	0.548	0.925	0.677	0.856	Success
Discrete Fourier transform		–	Fail	0.654	0.686	0.778	0.987	Success
Non-Overlapping template matching	m = 9	–	Fail	0.990	0.966	0.974	0.921	Success
The overlapping template matching test	B = 11111111	–	Fail	0.803	0.888	1	0.957	Success
Maurer' s universal statistical test		–	Fail	0.623	0.199	1	0.852	Success
Linear complexity test	M = 500	0.428	Success	0.861	0.825	0.818	0.642	Success
Serial test	m = 16	–	Fail	0.757	0.389	0.887	0.858	Success
Approximate entropy test	m = 10	–	Fail	0.739	0.846	0.598	0.871	Success
Cumulative sums		–	Fail	0.685	0.845	0.749	0.584	Success
Random Excursions	–	–						
x = 4	–	–	Fail	0.808	0.881	0.587	0.867	Success
x = 3	–	–	Fail	0.512	0.432	0.624	0.534	Success
x = 2	–	–	Fail	0.700	0.693	0.939	0.801	Success
x = 1	–	–	Fail	0.871	0.483	0.773	0.365	Success
x = 1	–	–	Fail	0.029	0.922	0.174	0.444	Success
x = 2	–	–	Fail	0.659	0.403	0.946	0.691	Success
x = 3	–	–	Fail	0.434	0.908	0.649	0.662	Success
x = 4	–	–	Fail	0.779	0.399	0.891	0.869	Success
Random Excursions Variant	–							
x = 9		–	Fail	0.439	0.728	0.477	0.919	Success
x = 9		–	Fail	0.416	0.852	0.452	0.872	Success
# of Passed Tests		2	–	15	15	15	15	–

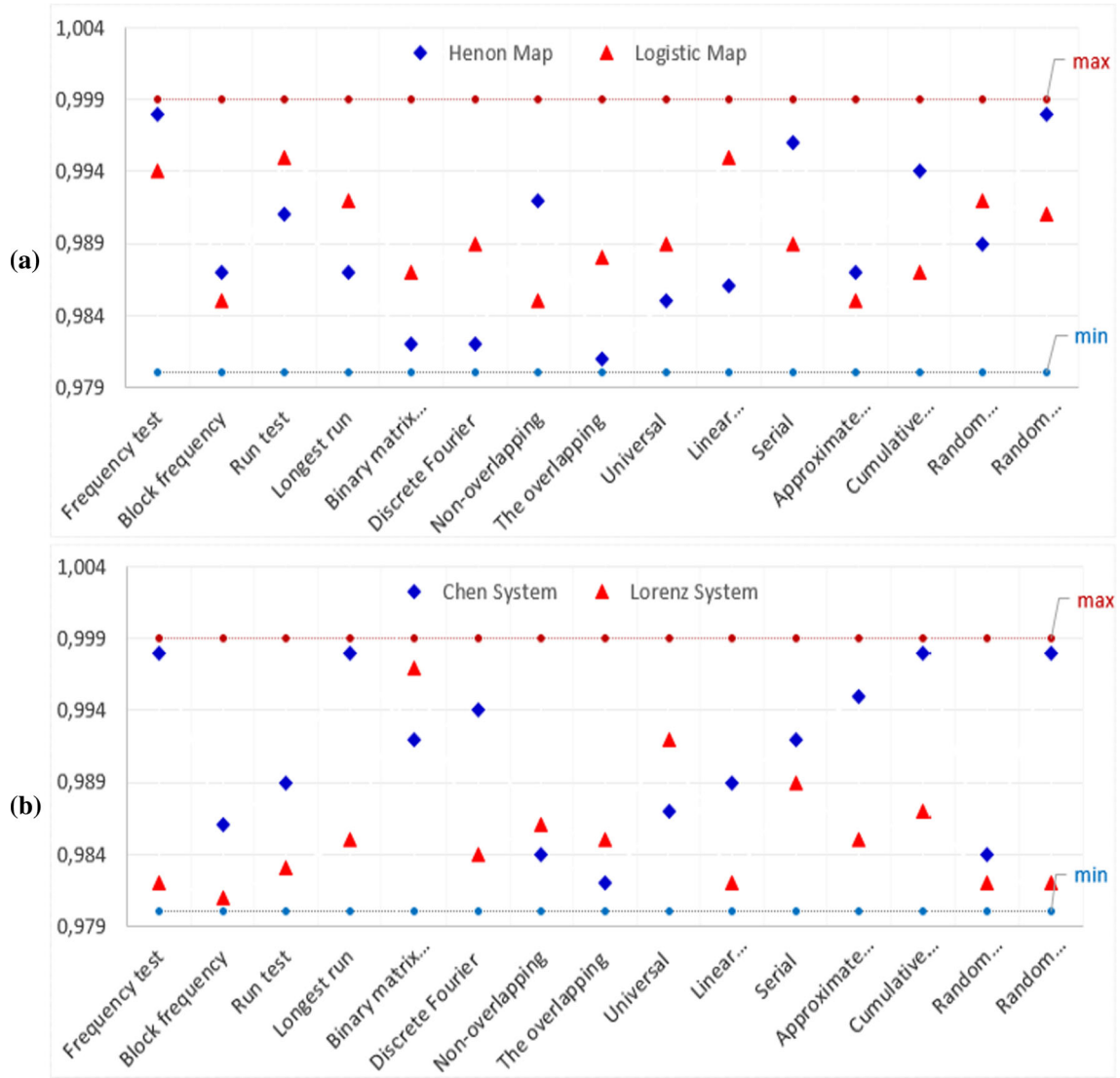


Fig. 14 All proportions of random numbers passing each test for the **a** first and **b** second scenarios

and the sample number is $m = 1.000$. With $a = 0.01$, it is calculated $conf.int._{[min,max]} = [0.9805, 0.9994]$ for each test criterion. Acceptable proportional values calculated for the post-processed random numbers are shown in Fig. 14.

$$\text{conf. int.}_{[\text{min, max}]} = \hat{p} \pm 3\sqrt{\frac{\hat{p}(1-\hat{p})}{m}} \tag{8}$$

The proportional values obtained for all test criteria for both scenarios are within the confidence interval. It is important that these proportional values calculated are within the confidence interval in order to make stronger inferences in terms of TRNG. Therefore, based on these results, we can clearly say that the post-processed TRNG outputs can be used for cryptographic purposes.

4.7 True randomness analysis

In the system, rising (positive) and falling (negative) edge-triggered two D-type flip flops ($DFF1\ and\ DFF2$ in Fig. 1) are used as in Fig. 1 (B) to sample the combined RO outputs. Although the data inputs and clock signal (clk) of these flip-flops are common, their sampling times are different. However, the true random oscillation of the common input applied to the data inputs and different sampling times make the d_0 and d_1 raw random numbers characteristically different from each other. Standard deviation analysis is used to verify this characteristic difference for d_0 and d_1 in the system and to show that these outputs are truly random. At this stage, TRNG is restarted 100 times and the d_0 and d_1 raw random number sequences obtained simultaneously for each restart experiment are recorded in two separate memory units. Then, these number sequences are grouped hex and the standard deviation analysis,



Fig. 15 Standard deviation distributions of d0 and d1 raw random sequences/outputs

whose mathematical definition is given in Eqs. 9–11, is applied. The μ and Var values are the weighted sum and variance values of the raw random number sequence whose standard deviation is to be calculated, respectively. $p(x_i)$ is the occurrence probability of four bits grouped random variables in each sequence. The standard deviation analysis results of d_0 and d_1 raw random number sequences are shown in Fig. 15.

$$\sigma = \sqrt{\mathrm{Var}(X)} \tag{9}$$

$$\mathrm{Var}(X) = \sum_{i=1}^{k} [x_i - E(X)]^2 . p(x_i) \tag{10}$$

$$\mu = E(X) = \sum_{i=1}^{k} x_i . p(x_i) \tag{11}$$

The results show that d_0 and d_1 raw random number sequences occur with a variable standard deviation distribution in the restart experiment. The fact that number sequences occur with different probabilistic distribution at the bit level with the natural effect of true randomness is the main reason for this variability. These results verifying the existence of true randomness in the system also confirm that the d_0 and d_1 raw random sequences are characteristically different from each other.

5 Performance analysis

5.1 Area–energy analysis

FPGA-based designs are evaluated with measurable features of these designs such as the highest accessible output bit rate, area and energy consumption. In addition to these basic criteria in cryptography, security priority complementary features such as resistance to physical attacks and aggressive tampering are also taken into consideration. Digital logic devices such as FPGAs consist of look-up table (LUT) and configurable logic blocks (CLB) formed by registers. The area requirement for designs is usually expressed in the number of CLB blocks. The number of these blocks is also of decisive importance in terms of energy consumption of the designs. Due to memory and time complexity, cryptographic applications may have high area–energy requirements at the hardware level. This is also true for TRNGs [45, 53].

Although the noise sources that can be used for a digital TRNG and the methods of obtaining randomness are limited, the post-processing preferences can be variable. At this point, security priority requirements that change according to the importance level of cryptographic applications are decisive. This variability also confirms that there is no ideal low-cost post-processing solution for TRNG offering high throughput bit rate. Post-processing techniques that are used especially to fulfill their statistical requirements can cause a significant cost (trade-off) problem. Low-performance (speed, randomness quality) high-cost post-processing preferences also limit the use of TRNG in lightweight architectures. However, for today's lightweight cryptographic solutions, there is a need for low complexity, simple and easily applicable, as well as cryptographically strong TRNG solutions. In this context, FPGA chip

Table 5 Device utilization summary of TRNG

Area/Resource Usage (Altera Cyclone IV GX: EP4CGX150DF31C8)	TRNG with proposed post-processing				Total FPGA Unit	
	Logistic & Lorenz		Henon and Chen		% Used	
	Used	Total	Used	Total	Available	Utilization
Total Logic Element Number	601	601	601	601	149.760	<1%
Combinational ALUTs	599	599	599	599	149.760	<1%
Dedicated Register Number	37	37	37	37	149.760	<1%
Total Memory Bits	0	0	0	0	6.635.520	0%
Total Pins	18	18	18	18	508	4%
Total PLLs	1	1	1	1	8	13%
Power Cons	125.04 mW		125.04 mW		–	–

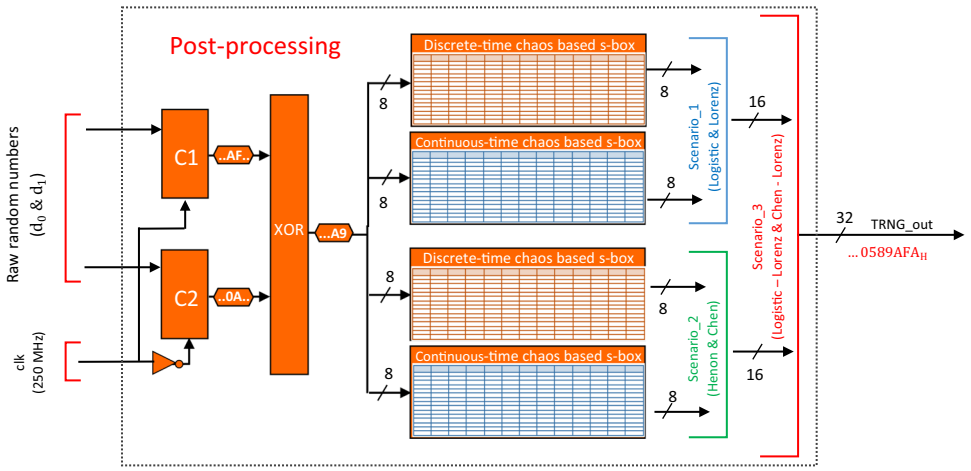


Fig. 16 Third post-processing scenario for TRNG

statistics for the TRNG architecture based on the post-processing technique suggested in two scenarios are shown in Table 5. The values are computed for typical operational conditions, where the junction temperature is 25 °C.

The complexity of the proposed post-processing method consisting of XOR and chaotic s-box tables is quite low. Especially the fact that s-boxes can be implemented as look-up tables on static RAM blocks further reduces this complexity at the hardware level. The proposed method's ability to fulfill the cryptographically needed statistical requirements at low cost in terms of area–energy consumption transforms TRNG into a lightweight model. Besides, when the data in Table 5 are examined, it can be said that TRNG is based on general principles applicable to all FPGA devices in terms of design parameters.

5.2 Throughput analysis

High-speed TRNG solutions are needed for today's modern cryptographic applications requiring the change of random numbers/key bits at the level of microseconds. Therefore, for a physical TRNG, speed is the next most important performance criterion after the security [4, 45]. Therefore, in terms of efficiency, a TRNG is asked to produce random numbers at minimum cost and high speed.

The output bit rate of a physical TRNG is directly dependent on the complexity and sampling rate as well as the compression amount of the post-processing techniques. The compression amounts of the post-processing techniques cause serious decreases in the output bit rate of TRNG, as well as increasing the production cost and entropy loss. The proposed post-processing technique is compressionless unlike known methods in the literature needing compression-based and costly solutions. In addition to compressionless feature, preference of two different s-box tables in the post-processing doubles the output bit rate of TRNG, because s-box tables in which 8-bit inputs are substituted are unique. So the outputs for the same raw vectorial inputs are completely different from each other.

The placement of the 8-bit row column values between 0 and 225 of the s-boxes used in the proposed post-processing is completely different from each other. Hence, for the same 8-bit raw random inputs, the s-box outputs representing the TRNG outputs are characteristically different from each other. In the study, it has been experimentally verified that these outputs analyzed independently for each scenario can successfully fulfil the statistical requirements associated with cryptographic security. In other respect, this makes it possible to apply four different s-boxes together in the proposed post-processing stage. The logical flowchart

Table 6 Literature comparison of TRNG based on the proposed post-processing

Study	Randomness Source	FPGA	Chip Area	Post-processing	Frequency (MHz)	Throughput (Mbps)
Ref. [3]	RO	Altera Cyclone IV	1309 LEs	AES s-box	250	250
Ref. [7]	RO	Altera Cyclone V	655 LEs	Resilient function	40	2.2
Ref. [8]	RO	Altera Cyclone II	83 LEs	–	100	100
Ref. [10]	Chaos	Xilinx Virtex-6	1318 Slices	XOR	464	464
Ref. [11]	ANN	Xilinx Virtex-6	131.202 Slices	XOR	167	167.4
Ref. [14]	Metastability	Altera Cyclone II	881 LEs	VN + LFSR (128-bit)	50	Non-declared
Ref. [16]	RO	Altera Cyclone IV	–	Chaos	450	20.45
Ref. [23]	Chaos	Xilinx Kintex-7	444 LEs	XOR	59.4	125 (kpbs)
Ref. [33]	Metastability	Altera Cyclone II	1.851 LEs	128 bit LFSR	50	26.6
Ref. [36]	RO	Xilinx Virtex-5	2.467 Slices	SHA-256	263	36
Ref. [41]	Chaos + Ro	Altera Cyclone IV	15.897 Les	Chaotic Entropy pool	450	34.6
Ref. [44]	RO	Xilinx Spartan 3	233 Slices	Cellular Automata	50	1.4
Ref. [51]	Chaos + RO	Altera Cyclone IV	2.011 LEs	–	200	15.4
Ref. [54]	RO	Altera Cyclone IV	715 LEs	XOR	200	61.64
Ref. [55]	PLL	Altera Cyclone II	1.334 LEs	Chaos + VN	70	26.84
Ref. [56]	Delay chain	Altera Stratix	–	VN	20	20.0
Ref. [57]	Chaos	Xilinx Virtex-6	12.462 Slices	XOR	293	58.76
Ref. [58]	Artificial Neural Network (ANN)	Xilinx Virtex–6	87.704 Slices	XOR	231	115.794
Ref. [59]	Metastability/Jitter	Xilinx Kintex	5 Slices	XOR	300	100
Ref. [60]	Metastability	Altera Cyclone III	511 LEs	XOR	Non-declared	133
Ref. [61]	Metastability	Xilinx Virtex-4	580 Slices	–	100	12.5
Ref. [62]	Metastability	Xilinx Spartan-6	60 Slices	VN	Non-declared	12.6
Ref. [63]	Dual Metastability	Xilinx Spartan–3E	–	VN	50	5.0
Ref. [64]	RO	Xilinx Spartan–3	270 Slices	VN	24	6.0
Ref. [65]	RO	Altera Cyclone II	899 LEs	VN + LFSR (128 bit)	27	27
Ref. [66]	Metastability	Xilinx Virtex -6	224 Slices	XOR	50	50
Ref. [67]	Metastability	Altera Cyclone IV	298 LEs	SHA-3 (Keccak)	150	150
Ref. [68]	Chaos	Xilinx Spartan 6	256 LEs	Non-declared	125	125
Ref. [69]	Non-Linear Feedback RO	Altera Cyclone IV	137 LEs	8-bit LFSR	200	200
Ref. [70]	Self-Timed Rings (STRs)	Xilinx Spartan-6	56 LEs	XOR + VN	100–400	100
Ref. [71]	Galois and Fibonacci RO	Xilinx Zynq	866 Slices	XOR + VN	40	6.25
Ref. [72]	RO + Metastability	Altera Cyclone IV	2066 LEs	DES s-box	200	133.3
TRNG	RO	Altera Cyclone IV	601 LEs	Chaos based s-boxes	250	1 (Gbps)

diagram of a third post-processing scenario for TRNG in which both scenarios are combined (four different chaotic s-box tables are applied together) is shown in Fig. 16.

The raw random numbers sampling rate of TRNG is 250 Mbit/s. In the first two scenarios, TRNG produces two different 8-bit random outputs (16 bits in total) substituted in different s-boxes for the same 8-bit raw random inputs. Therefore, for the first two scenarios, the final output bitrate of TRNG is twice the sampling rate, i.e., 500 Mbit/s. The output bitrate is equal to four times the sampling rate for the third scenario in Fig. 16, where both scenarios are combined, i.e., 1 Gbit/s. Therefore, TRNG is quite successful in terms of output bit rate, which is another important evaluation parameter, compared to other studies in the literature. Comparison results are shown in Table 6.

Increasing the sampling rate of raw random numbers worsens the statistical distribution of these numbers, leading to a trade-off between output bit rate and the randomness quality. However, the proposed method is based on a cryptographically strong atomic basis such as s-box, eliminating this handicap. Also, the proposed method is low in complexity as it is based on a simple fictional basis. Therefore, it can provide high output bit rate for TRNG with low area–energy requirement. Thus, TRNG whose good statistical properties are guaranteed with lower hardware cost by the proposed method can be used in different embedded platforms and applications such as multi-core processors, mobile devices, Internet of Things (IoT), sensor networks, radio frequency identification

(RFID) tags and smart cards. Comparison results of TRNG also show that the proposed method provides better trade-off than the methods in the literature in terms of speed, cost, security and randomness quality.

6 Conclusions

In this study, FPGA-based implementation and performance analysis of a novel high-performance lightweight post-processing technique that can successfully eliminate the statistical weakness problem of TRNGs are presented. The effectiveness of the method is tested in two different scenarios by applying it to an RO-based TRNG architecture in real time. The obtained test results show that TRNG can successfully pass these tests with an output bit rate of 1 Gbit/s with the effect/contribution of the post-processing. In addition, it has been observed that TRNG for the proposed method has a cryptographically higher workload than the methods and architectures in the literature, with the low area–energy requirement. This shows that besides the cryptographic suitability of the post-processing method, TRNG can generate random numbers at high speed at low cost without compromising security. The results also prove that the proposed post-processing method successfully meets the lightweight requirements for cryptographic convenience.

The fact that the chaos-based s-box architectures selected for the proposed method can offer good statistical capabilities for TRNG confirms the basic idea of the study. Besides the low complexity and area–energy requirement, flexibility is another prominent feature of the proposed method. Flexibility and ease of modeling make it possible to develop new scenarios based on different chaotic s-boxes in the future. This also makes the method testable in different noise-based TRNGs. In another aspect, the study offers a different perspective that can be a source for future lightweight and TRNG-based cryptographic studies.

Acknowledgements Authors are deeply grateful to the editor(s) for smooth and fast handling of the manuscript. The authors would also like to sincerely thank the anonymous reviewer(s) for their valuable contributions to improve the quality of this paper.

Appendix A

The s-box tables for the second scenario are as follows:

See Tables 7 and 8.

Table 7 Henon map based s-box matrix/table

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	63	A1	9F	98	82	6C	EA	5A	FC	F0	C2	28	55	CC	39	51
1	95	CE	D6	58	0F	3E	37	69	74	3D	53	E1	4A	87	76	DA
2	F9	86	7E	01	02	E3	2C	48	E5	34	C7	1D	E2	AC	45	EE
3	CD	07	2D	20	BB	0A	35	4C	15	1A	AF	6B	92	AB	62	A9
4	C8	23	27	43	6E	03	71	AA	7D	05	A5	70	9B	C6	A3	EC
5	FE	61	5B	7B	A8	60	DE	F1	7C	1B	44	D4	FB	8D	81	66
6	DF	47	D7	3B	EF	22	D3	2B	6D	7A	04	D5	30	90	E4	9E
7	D9	E8	9C	F2	BC	57	93	1C	7F	72	2A	65	54	88	D1	40
8	1F	FD	64	12	B8	5D	E7	0C	78	33	DC	C0	F4	F5	CA	84
9	3F	96	FA	09	8E	36	C1	91	3C	B9	31	D2	32	41	6F	1E
10	ED	97	B5	2F	73	8F	A0	F6	46	5E	BA	94	B4	BD	3A	F7
11	6A	18	D0	AE	9D	89	52	0E	DB	9A	80	19	16	4B	29	24
12	8B	08	EB	A4	8C	F8	25	8A	B6	BF	79	FF	D8	B1	0B	4F)
13	A7	17	49	A2	68	56	A6	B2	21	85	4E	38	83	BE	B7	2E
14	42	4D	B3	DD	77	B0	00	E0	CB	C4	E6	67	13	C9	E9	5C
15	AD	10	C3	C5	14	CF	06	50	5F	59	75	0D	99	F3	11	26

Table 8 Chen system-based s-box matrix/table

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0B	0C	37	9C	61	88	5C	82	B7	9F	59	9E	B8	0D	17	39
1	63	F5	5D	F2	A0	74	F9	8E	92	8D	1C	E2	F4	4E	45	70
2	55	B2	CF	E7	6E	87	07	3A	CA	EF	64	2D	81	DC	71	EE
3	6C	66	D2	33	C1	30	E6	C2	15	5F	F8	6F	F6	C0	F3	CC
4	27	F7	EC	84	23	DA	3D	58	DE	26	2F	86	E3	EB	A6	C9
5	FC	06	B4	57	8A	10	90	68	69	83	1A	CB	3B	5B	40	CE
6	8B	7F	C6	3E	12	32	46	50	49	AF	35	47	4C	A1	DD	FE
7	28	D3	B5	DB	EA	25	AA	77	2B	80	FF	97	F1	BD	0A	7B
8	C3	1B	DF	CD	D9	C5	1E	C8	BC	31	65	D8	4B	A2	19	3F
9	79	3C	54	22	A4	95	BB	AB	7E	B0	1F	BF	02	A5	D4	8F
10	43	7D	13	E0	51	04	D0	AE	34	76	2C	29	42	94	0E	FA
11	E1	96	91	B9	09	89	E8	4D	75	A8	B6	FB	A7	24	00	48
12	F0	D6	4A	60	14	BE	9A	D5	D7	6A	D1	C4	99	5A	41	52
13	FD	B1	73	A9	16	E9	78	9D	44	2A	01	85	03	A3	21	38
14	B3	53	36	C7	4F	6D	BA	7A	0F	05	72	93	2E	E4	AD	5E
15	20	67	E5	6B	ED	9B	62	7C	AC	8C	18	11	56	1D	08	98

References

1. L. Gong, J. Zhang, H. Liu, L. Sang, Y. Wang, True random number generators using electrical noise. IEEE Access **7**, 125796–125805 (2019)
2. Ç. Koç, *About Cryptographic Engineering* (Springer, Boston, MA, 2009)
3. A.M. Garipcan, E. Erdem, Design, FPGA implementation and statistical analysis of a high-speed and low-area TRNG based on an AES S-box post-processing technique. ISA Transactions (2021)
4. V. Fischer, A closer look at security in random number generators design. Lecture Notes in Computer Science, pp. 167–182, 2012 (2012). Doi: https://doi.org/10.1007/978-3-642-29912-4_13
5. D. Rushchen, M. Schrey, J. Freese, I. Heisterklaus, Generation of true random numbers based on radioactive decay. In Proceedings of the International Student Scientific Conference Poster—21/2017. Prague (Czech Republic), pp. 1–4 (2017)
6. B. Jun, P. Kocher, The Intel random number generator. Cryptograph Research Inc., White Paper, vol. 27, pp. 1–8 (1999)
7. B. Sunar, W.J. Martin, D.R. Stinson, A provably secure true random number generator with built-in tolerance to active attacks. IEEE Trans. Comput. **56**, 109–119 (2007)
8. K. Wold, C.H. Tan, Analysis and enhancement of random number generator in FPGA based on oscillator rings. Int. J. Reconfigurable Comput. **2009**, 385–390 (2009)
9. D. Lambić, A. Janković, M. Ahmad, Security analysis of the efficient chaos pseudo-random number generator applied to video encryption. J. Electron. Test. **34**(6), 709–715 (2018)
10. İ. Koyuncu, M. Tuna, İ. Pehlivan, C.B. Fidan, M. Alçın, Design, FPGA implementation and statistical analysis of chaos-ring based dual entropy core true random number generator. Analog Integrated Circuits and Signal Processing, pp. 1–12 (2019)
11. M. Tuna, A. Karthikeyan, K. Rajagopal, M. Alcin, İ Koyuncu, Hyperjerk multiscroll oscillators with megastability: analysis, FPGA implementation and a novel ANN-ring-based true random number generator. AEU-Int. J. Electron. Commun. **112**, 152941 (2019)
12. M. Ahmad, M.N. Doja, M.M.S. Beg, Security analysis and enhancements of an image cryptosystem based on hyperchaotic system. J. King Saud Univ.-Comput. Inf. Sci. **33**(1), 77–85 (2021)
13. H. Hata, S. Ichikawa, FPGA implementation of metastability-based true random number generator. IEICE Trans. Inf. Syst. **95**(2), 426–436 (2013)
14. P.S. Meka, R. Sivaraman, A. Rengarajan, S. Rajagopalan, Metastability Influenced PUF for cryptographic key generation: a FPGA Approach. In 2020 International conference on computer communication and informatics (pp. 1–6) (2020)
15. M. Stipčević, Ç.K. Koç, True random number generators, in *Open Problems in Mathematics and Computational Science*. (Springer, Cham, 2014), pp. 275–315
16. E. Avaroğlu, T. Tuncer, A.B. Özer, B. Ergen, M. Türk, A novel chaos-based post-processing for TRNG. Nonlinear Dyn. **81**(1–2), 189–199 (2015)
17. F. Özkaynak, Cryptographically secure random number generator with chaotic additional input. Nonlinear Dyn. **78**(3), 2015–2020 (2014)
18. S. Toprak, A. Akbulut, M.A. Aydın, A.H. Zaim, LWE: An energy-efficient lightweight encryption algorithm for medical sensors and IoT devices. Electrica **20**(1), 71–81 (2020)
19. K. McKay, L. Bassham, M. Sönmez Turan, N. Mouha, Report on lightweight cryptography. Draft NIST Internal or Interagency Report (NISTIR) 8114, Tech. Rep., National Institute of Standards and Technology, August 2016 (2016)
20. V. Rožić, I. Verbauwhede, Hardware-efficient post-processing architectures for true random number generators. IEEE Trans. Circuits Syst. II Express Briefs **66**(7), 1242–1246 (2018)
21. K. Marton, A. Suciu, I. Ignat, Randomness in digital cryptography: a survey. Romanian J. Inf. Sci. Technol. **13**(3), 219–240 (2018)
22. R.B. Davies, Exclusive OR (XOR) and hardware random number generators. March 07, 2021, Available at http://www.robertnz.net/pdsf/xor2.pdf
23. B. Karakaya, A. Gülten, M. Frasca, A true random bit generator based on a memristive chaotic circuit: analysis, design and FPGA implementation. Chaos Solitons Fractals **119**, 143–149 (2019)
24. M. Dichtl, Bad and good ways of post-processing biased physical random numbers. In Fast Software Encryption workshop—FSE, 4593: 127–152 (2007)
25. A.M. Garipcan, E. Erdem, Hardware implementation of chaotic zigzag map based bitwise dynamical PRNG on FPGA. Informacije MIDEM **50**(4), 243–254 (2021)
26. S. Nikolic, M. Veinovic, Advancement of true random number generators based on sound cards through utilization of a new post-processing method. Wireless Pers. Commun. **91**(2), 603–622 (2016)

27. J. Von Neumann, various techniques used in connection with random digits. National Bureau Stand. Appl. Math. Series **12**, 36–38 (1951)
28. R. Zhang, S. Chen, C. Wan, H. Shinohara, High-throughput Von Neumann post-processing for random number generator. In 2018 International Symposium on VLSI Design, Automation and Test (VLSI-DAT) IEEE, pp. 1–4 (2018)
29. V. Rožić, B. Yang, W. Dehaene, & I. Verbauwhede, Iterating von Neumann's post-processing under hardware constraints. In 2016 IEEE international symposium on hardware oriented security and trust (HOST), pp. 37–42 (2016)
30. A. Degada, H. Thapliyal, An integrated TRNG-PUF architecture based on photovoltaic solar cells. IEEE Consumer Electr. Magazine (2020). https://doi.org/10.1109/MCE.2020.3019762
31. D. Schellekens, B. Preneel, I. Verbauwhede, FPGA vendor agnostic true random number generator. In: Proc. 16th Int. Conf. Field Programmable Logic and Applications—FPL, pp. 1–6 (2006)
32. J.D. Golic, New methods for digital generation and postprocessing of random data. IEEE Trans. Comput. **55**(10), 1217–1229 (2006)
33. R. Sivaraman, S. Rajagopalan, A. Sridevi, J.B.B. Rayappan, M.P.V. Annamalai, A. Rengarajan, Metastability-induced TRNG architecture on FPGA. Iranian J. Sci. Technol. Trans. Electr. Eng. **44**(1), 47–57 (2020)
34. M. Dichtl, How to predict the output of a hardware random number generator, in *International Workshop on Cryptographic Hardware and Embedded Systems*. (Springer, Berlin, Heidelberg, 2003), pp. 181–188
35. V. Rožić, I. Verbauwhede, Hardware-efficient post-processing architectures for true random number generators. IEEE Trans. Circuits Syst. II Express Briefs **66**(7), 1242–1246 (2018)
36. S. Łoza, & Ł. Matuszewski, A true random number generator using ring oscillators and SHA-256 as post-processing. In 2014 International Conference on Signals and Electronic Systems (ICSES) IEEE, pp. 1–4 (2014)
37. S. Yakut, T. Tuncer, A.B. Ozer, Secure and efficient hybrid random number generator based on sponge constructions for cryptographic applications. Elektronika ir Elektrotechnika **25**(4), 40–46 (2019)
38. K. Márton, L. Pârvu, A. Suciu, The Impact of Post-processing Functions on Random Number Sequences. In 2018 17th RoEduNet Conference: Networking in Education and Research (RoEduNet) IEE pp. 1–6 (2018)
39. V.B. Suresh, W.P. Burleson, Entropy and energy bounds for metastability based TRNG with lightweight post-processing. IEEE Trans. Circuits Syst. I Regul. Pap. **62**(7), 1785–1793 (2015)
40. V. Fischer, F. Bernard, & N. Bochard, Modern random number generator design–Case study on a secured PLL-based TRNG, it-Information Technology, cilt 61(1), ss. 3–13 (2019)
41. A.M. Garipcan, E. Erdem, A TRNG using chaotic entropy pool as a post-processing technique: analysis, design and FPGA implementation. Analog Integr. Circ. Sig. Process **103**(3), 1–20 (2020)
42. J.S. Teh, W. Teng, A. Samsudin, J. Chen, A post-processing method for true random number generators based on hyperchaos with applications in audio-based generators. Front. Comp. Sci. **14**(6), 1–11 (2020)
43. J.J.M. Chan, P. Thulasiraman, G. Thomas, R. Thulasiram, Ensuring quality of random numbers from TRNG design and evaluation of post-processing using genetic algorithm. J. Comput. Commun. **4**(4), 73–92 (2016)
44. J. Cartagena, H. Gomez, & E. Roa, A fully-synthesized TRNG with lightweight cellular-automata based post-processing stage in 130nm CMOS. In 2016 IEEE Nordic Circuits and Systems Conference (NORCAS) (pp. 1–5). IEEE (2016)
45. V. Fischer, M. Deutschmann, S. Lattacher, G. Battum, Report on Selected TRNG and PUF Principles. HECTOR Project Technichal Report D2.1, Université Jean Monnet (UJM) (2016)
46. F. Özkaynak, Construction of robust substitution boxes based on chaotic systems. Neural Comput. Appl. **31**(8), 3317–3326 (2019)
47. M.P.S. dos Santos, J.A.F. Ferreira, Novel intelligent real-time position tracking system using FPGA and fuzzy logic. ISA Trans. **53**(2), 402–414 (2014)
48. A. Rukhin, J. Soto, J. Nechvatal, M. Smid, D. Banks, A statistical test suite for random and pseudorandom number generators for statistical applications. NIST Special Publication in Computer Security (2001)
49. A.J. Menezes, P.C. Oorschot, S.A. Vanstone, *Handbook of Applied Cryptography* (CRC Press, Boca Raton, FL, USA, 1996)
50. P.W. Kohlbrenner, The design and analysis of a true random number generator in a field programmable gate array. Doctoral dissertation, George Mason University (GMU) (2003)
51. A.M. Garipcan, E. Erdem, Implementation and performance analysis of true random number generator on FPGA environment by using non-periodic chaotic signals obtained from chaotic maps. Arab. J. Sci. Eng. **44**(11), 9427–9441 (2019)
52. M. Ahmad, , Farooq, O. (2011). Chaos based PN sequence generator for cryptographic applications. In 2011 International Conference on Multimedia, Signal Processing and Communication Technologies (pp. 83–86). IEEE
53. T.L. Liao, P.Y. Wan, J.J. Yan, Design and synchronization of chaos-based true random number generators and its FPGA implementation. IEEE Access (2022)
54. A.M. Garipcan, E. Erdem, Implementation of a digital TRNG using jitter based multiple entropy source on FPGA. Informacije MIDEM **49**(2), 79–90 (2019)
55. S. Rethinam, S. Rajagopalan, S. Janakiraman, S. Arumugham, R. Amirtharaian, Jitters through dual clocks: an effective Entropy Source for True Random Number Generation. In 2018 International Conference on Computer Communication and Informatics (ICCCI), pp. 1–5 (2018)
56. J.-L. Danger, S. Guilleya, P. Hoogvorsta, High speed true random number generator based on open loop structures in FPGAs. Microelectron. J. **40**(11), 1650–1656 (2009)
57. I. Koyuncu, A.T. Ozcerit, The design and realiza¬tion of a new high speed FPGA-based chaotic true random number generator. Comput. Electr. Eng. **58**, 203–214 (2017)
58. M. Alcin, I. Koyuncu, M. Tuna, M. Varan, I. Pehlivan, A novel high speed artificial neural network–based chaotic true random number generator on field programmable gate array. Int. J. Circuit Theory Appl. **47**(3), 365–378 (2019)
59. G.D.P. Stanchieri, A. De Marcellis, E. Palange, M. Faccio, A true random number generator architecture based on a reduced number of FPGA primitives. AEU-Int. J. Electr. Commun. **105**, 15–23 (2019)
60. A. Cherkaoui, V. Fischer, L. Fesquet, A. Aubert, A very high speed true random number generator with entropy assessment. In International Conference on Cryptographic Hardware and Embedded Systems, pp. 179–196 (2013)
61. H. Hata, S. Ichikawa, FPGA implementation of metastability-based true random number generator. IEICE Trans. Inf. Syst. **95**(2), 426–436 (2012)
62. P.Z. Wieczorek, An FPGA implementation of the resolve time-based true random number generator with quality control. IEEE Trans. Circuits Syst. I Regul. Pap. **61**(12), 3450–3459 (2014)
63. P.Z. Wieczorek, K. Gołofit, Dual-metastability time-competitive true random number generator. IEEE Trans. Circuits Syst. I Regul. Pap. **61**(1), 134–145 (2013)
64. N.N. Anandakumar, S.K. Sanadhya, M.S. Hashmi, FPGA-based true random number generation using programmable delays in oscillator-rings. IEEE Trans. Circuits Syst. II: Express Briefs **67**(3), 570–574 (2019)

65. R. Sivaraman, S. Rajagopalan, R. Amirtharajan, FPGA based generic RO TRNG architecture for image confusion. Multimedia Tools Appl., pp.1–28 (2020)
66. Yang X., Cheung R.C.C., A complementary architecture for highspeed true random number generator. In 2014 International Conference on Field-Programmable Technology (FPT), pp. 248–251, IEEE (2014)
67. X. Wu, S. Li, A new digital true random number generator based on delay chain feedback loop. In 2017 IEEE International Symposium on Circuits and Systems (ISCAS), pp. 1–4 (2017)
68. Y. Yang, S. Jia, Y. Wang, A reliable true random number generator based on novel chaotic ring oscillator. In 2017 IEEE International Symposium on Circuits and Systems (ISCAS), pp. 1–4, IEEE (2017)
69. S. Tao, Y. Yu, E. Dubrova, FPGA based true random number generators using non-linear feedback ring oscillators. In 2018 16th IEEE International New Circuits and Systems Conference (NEWCAS) IEEE, pp. 213–216 (2018)
70. X. Wang, H. Liang, Y. Wang, L. Yao, Y. Guo, M. Yi, Y. Lu, High-throughput portable true random number generator based on jitter-latch structure. IEEE Trans. Circuits Syst. I: Regular Papers (2020)
71. K. Demir, S. Ergun, Random number generators based on irregular sampling and fibonacci–galois ring oscillators. IEEE Trans. Circuits Syst. II Express Briefs **66**(10), 1718–1722 (2019)
72. A.M. Garipcan, E. Erdem, DESSB-TRNG: a novel true random number generator using data encryption standard substitution box as post-processing. Digital Signal Process. **123**, 103455 (2022)